DEPARTMENT OF INFORMATICS

TECHNICAL UNIVERSITY OF MUNICH

Bachelor's Thesis in Informatics

Combining Word Embeddings Of Protein Sequences With Evolutionary Information For Secondary Structure Prediction Using Deep Neural Networks

Anna Reithmeir

DEPARTMENT OF INFORMATICS

TECHNICAL UNIVERSITY OF MUNICH

Bachelor's Thesis in Informatics

Combining Word Embeddings Of Protein Sequences With Evolutionary Information For Secondary Structure Prediction Using Deep Neural Networks

Kombination von Vektor-Repräsentationen von Proteinsequenzen und Evolutionärer Information zur Vorhersage von Proteinstruktur-Eigenschaften unter Verwendung von Neuronalen Netzwerken

Author: Anna Reithmeir
Supervisor: Prof. Dr. Burkhard Rost
Advisor: M. Sc. Michael Heinzinger
Submitted: 15.04.2019

I confirm that this bachelor's thesis in Informatics is my own work and I have documented all sources and material used.

Munich, 15.04.2019 Anna Reithmeir

Abstract

The identification of the three-dimensional structure of a protein is fundamental since it determines the protein's function. In bioinformatics, neural networks have proven to be useful for the prediction of the secondary structure from the linear amino acid sequence. Here, evolutionary information of the protein is crucial, which is given by a multiple sequence alignment (MSA). However, the complexity of computing such MSAs poses a challenge, which is further intensified by the continuing growth of sequence databases. Hence, a method which does not rely on the computation of MSAs is desirable. Word embeddings, which in natural language processing represent similar words by similar vectors, could serve as such an alternative. ProtVec embeddings [1] have been trained on the SwissProt database and encode combinations of amino acids as vectors. These implicitly hold evolutionary information since amino acids with similar biochemical properties are represented by similar vectors.
In this thesis, it is investigated to what extent the use of ProtVec embeddings can improve the performance of deep neural networks in secondary structure prediction. Moreover, it will be studied if the prediction can benefit from the additional learning of structural features, namely the relative solvent accessibility and the flexibility. To this end, six different network architectures are trained on a data set of 2935 protein sequences and the resulting predictions are then evaluated. Apart from one convolutional neural networks (CNN), two long short-term memory networks (LSTMs), one combination of the two network types and two networks with DenseNet-inspired layers are employed.
The experiments show that the simple encoding of an amino acid sequence through ProtVec embeddings does not improve the prediction accuracy. However, the employed DenseHybrid architecture reaches a Q3 accuracy of 83.17% when a combination of the position-specific scoring matrix and the ProtVec encoded MSA is used as the input. Finally, the results show that the learning of additional structural features is possible without a significant loss of performance.

Zusammenfassung

Die Identifizierung der räumlichen Struktur eines Proteins ist essentiell, da dessen Funktion durch seine dreidimensionale Konfiguration bestimmt wird. In der Bioinformatik haben sich neuronale Netze als nützlich erwiesen, um die Sekundärstruktur aus der linearen Aminosäuresequenz vorherzusagen. Hierbei ist die Verwendung von evolutionärer Information des Proteins entscheidend, die in multiplen Sequenzalignments (MSAs) gegeben ist. Jedoch stellt die Komplexität der Berechnung von MSAs eine Herausforderung dar, welche durch das konstante Wachstum der Sequenzdatenbanken weiter verstärkt wird. Infolgedessen wird nach Methoden der Strukturvorhersage gesucht, die nicht auf die Berechnung von MSAs angewiesen sind. Vektor-Repräsentationen, die in der natürlichen Sprachverarbeitung (Natural language processing) ähnliche Wörter als ähnliche Vektoren darstellen, könnten eine Alternative bieten. ProtVec Vektor-Repräsentationen [1] wurden auf der SwissProt Datenbank trainiert und kodieren Kombinationen von Aminosäuren als Vektoren. Evolutionäre Information ist implizit enthalten, da biochemisch ähnliche Aminosäuren ähnlichen ProtVec Vektoren zugewiesen sind.

In dieser Bachelorarbeit wird untersucht, ob die Verwendung von ProtVec Vektoren einen positiven Einfluss auf die Sekundärstrukturvorhersage durch neuronale Netze hat. Außerdem wird die Frage behandelt, inwiefern das zusätzliche Lernen von Oberflächenzugänglichkeit und Flexibilität der einzelnen Residuen die Vorhersage verbessern kann. Zu diesem Zweck werden sechs neuronale Netze auf einem Datensatz, bestehend aus 2935 Proteinen, trainiert und die Vorhersagen anschließend ausgewertet. Neben einem 'Convolutional Neural Network' (CNN) werden zwei 'Long Short-Term Memory' Netzwerke (LSTMs), eine Kombination aus beiden und zwei Netzwerke mit 'DenseNet'-inspirierten Layern implementiert.

Unsere Experimente zeigen, dass das einfache Kodieren der Aminosäuresequenz durch ProtVec Vektor-Repräsentationen keine Verbesserung in der Genauigkeit der Strukturvorhersage erzielt. Dennoch konnte unser DenseHybrid-Netzwerk mit Hilfe der Repräsentation eines Proteins durch die Kombination aus der positionsspezifischen Gewichtsmatrix und der ProtVec Repräsentation des MSAs eine Q3 Vorhersagegenauigkeit von 83.17% erreichen. Die Auswertung der Ergebnisse zeigt zudem, dass das Lernen von zusätzlichen Struktureigenschaften ohne signifikanten Verlust der Vorhersagegenauigkeit möglich ist.

Contents

Abstract **iii**

Zusammenfassung **v**

List of Figures **ix**

List of Tables **xi**

List of Abbreviations **xiii**

1 Introduction **1**
1.1 Problem Statement and Outline 1
1.2 Background 2
1.2.1 Structural Features of Proteins 2
1.2.2 Evolutionary Information 3
1.2.3 Machine Learning Techniques 4
1.2.4 ProtVec Word Embeddings 6
1.3 Related Work 7

2 Implementation **9**
2.1 Data Analysis 9
2.2 The Overall Workflow 11
2.2.1 Data Pre-Processing 11
2.2.2 Network Architectures 14
2.2.3 Multi-Task Learning 16
2.2.4 Overall Prediction Model 16
2.3 Training Parameters and Training Procedure 17

3 Experimental Evaluation **19**
3.1 Performance Metrics 19
3.2 Results 20
3.2.1 Effect of ProtVec for the CNN Architecture 20
3.2.2 Performance of the Different Network Architectures 23
3.2.3 Effect of Multi-Task Learning 25
3.2.4 Effect of Mapping Predictions from DSSP8 to DSSP3 Structures 27
3.2.5 DSSP8 Classification 27
3.2.6 Effect of ProtVec for the DenseHybrid Architecture 29

3.3 Runtime and Number of Parameters 30

4 Discussion **33**

5 Conclusion and Outlook **37**

Bibliography **39**

A Supplementary Material
A.1 Data Analysis of the Predicted DSSP3 Structures . . . . . .
A.2 Loss Curves . . . . . .

List of Figures

1.1 Growth of UniProtKB/SwissProt and PDB Databases 1
1.2 Multi-Task Learning Model 6

2.1 Distribution of Sequence Lengths 10
2.2 Class Distribution of DSSP Structures 11
2.3 Distribution of Consecutively Occurring States in DSSP3 12
2.4 Distribution of RSA Values and B-Factors 12
2.5 CNN Architecture 14
2.6 DenseCNN Architecture 15
2.7 Overall Model 17

3.1 Learning Curves of CNN for Different Inputs 21
3.2 Confusion Matrices of CNN (no Evolutionary Information) 21
3.3 Confusion Matrices of CNN (with Evolutionary Information) 23
3.4 Learning Curves of All Networks 24
3.5 Learning Curves of the DenseHybrid Network for the Multi-Task Learning Variants 25
3.6 Prediction of RSA Values and B-Factors for a Random Protein 26
3.7 Learning Curve of the DenseHybrid Network for DSSP8 Classification . 28
3.8 Confusion Matrix of the DenseHybrid Network For DSSP8 Classification 28
3.9 Learning Curves of the DenseHybrid Network for Different Inputs . . . 30

A.1 Distribution of the Predicted DSSP Structures . . . . . .
A.2 Distribution of Consecutively Occurring States in DSSP3 Predictions .
A.3 Loss Curves of the CNN for Different Inputs . . . . . .
A.4 Loss Curves of All Network Architectures . . . . . .
A.5 Loss Curves of the DenseHybrid Architecture for the Multi-Task Learning Variants . . . . . .
A.6 Loss Curve of the DenseHybrid Architecture for DSSP8 Classification .
A.7 Loss Curves of the DenseHybrid Architecture for Different Inputs . . .

List of Tables

1.1 DSSP Secondary Structure States 3

2.1 Mapping the DSSP Structures to Numerical Values 13
2.2 Learning Rates 18

3.1 Class-Specific Measurement Results of CNN (no Evolutionary Information) 22
3.2 Class-Specific Measurement Results of CNN (with Evolutionary Information) 23
3.3 Class-Specific Measurement Results of All Networks 24
3.4 Class-Specific Measurement Results of DenseHybrid Network for the Multi-Task Learning Variants 26
3.5 Class-Specific Measurement Results for DenseHybrid Network when Mapping DSSP8 to DSSP3 Predictions 27
3.6 Class-Specific Measurement Results of DenseHybrid Network for DSSP8 Classification 29
3.7 Performance of the DenseHybrid Network for Different Inputs 30
3.8 Runtime and Number of Parameters 31

List of Abbreviations

3D	**3**-**D**imensional
BiLSTM	**Bi**directional **L**ong **S**hort-**T**erm **M**emory
CNN	**C**onvolutional **N**eural **N**etwork
DNA	**D**eoxyribo**N**ucleic **A**cid
DNN	**D**eep **N**eural **N**etwork
LSTM	**L**ong **S**hort-**T**erm **M**emory
MSA	**M**ultiple **S**equence **A**lignment
MTL	**M**ulti-**T**ask **L**earning
PDB	**P**rotein **D**ata **B**ase
PSSM	**P**osition-**S**pecific **S**coring **M**atrix
RNN	**R**ecurrent **N**eural **N**etwork
RSA	**R**elative **S**olvent **A**ccessibility

Chapter 1

Introduction

1.1 Problem Statement and Outline

Protein secondary structures can be determined with X-ray crystallography, nuclear magnetic resonance spectroscopy and cryo-electron microscopy [2]. However, the number of identified protein structures can not keep up with the number of newly discovered protein sequences due to the complexity of the required experiment. Thus the protein sequence - structure gap is continuing to be a central issue and will be widening further in the future. At the time of writing, the UniProtKB/SwissProt database holds in total 558 898 protein entries while the Research Collaboratory for Structural Bioinformatics Protein Data Base (PDB) only holds 147 073 protein sequence entries, see Figure 1.1. Because the structure directly determines the biological function of a protein, structural knowledge plays an essential role in various fields, e.g. in drug design [3], development of vaccines [4] or protein engineering [5]. In order to keep up, the prediction of the three-dimensional (3D) structure from the linear amino acid sequence has become a fundamental problem in bioinformatics.

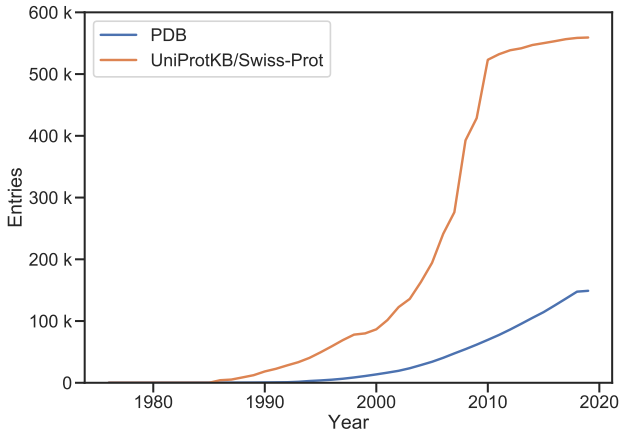


FIGURE 1.1: Growth of UniProtKB/SwissProt and PDB databases. The data is taken from [6] and [7].

Meanwhile, deep neural networks (DNNs) have successfully been applied to protein structure prediction, e.g., see [8]–[11]. New models are being proposed continuously and Q3 accuracies of up to 85% have recently been reached [12]. As the task of predicting a sequence of structure patterns to a given amino acid sequence has similarities to machine translation in natural language processing (NLP), it is interesting to investigate how methods successfully used in NLP can be adapted to the field of bioinformatics. One NLP technique is the representation of the input through word embeddings, where similar words share similar representations. This is realized in the ProtVec [1] approach. Having been trained on protein sequences, ProtVec embeddings provide fixed-sized continuous vector representations of 3-grams of amino acids. These implicitly hold evolutionary information because biochemically similar amino acids possess similar vectors. This work will investigate two questions regarding ProtVec embeddings:

- Can the ProtVec embeddings serve as a computationally efficient method to provide evolutionary information to a DNN? This would be beneficial, since the complexity to compute MSAs is increasing due to continuously growing databases.
- Can the performance of a DNN be increased when providing the ProtVec embeddings alongside other evolutionary information?

To this end, we implemented six neural network architectures and evaluated them on five kinds of input representations. Furthermore, we examined the effect of learning different structural features simultaneously with hard parameter shared multi-task learning.

The remainder of this thesis is organized as follows: First, in Chapter 1, background information on protein secondary structure, DNNs, multi-task learning and word embeddings is given and related work is discussed. In Chapter 2, the details of our implementation of the prediction model are explained. In Chapter 3, the experimental results are presented. In Chapter 4, our results are discussed and, finally, in Chapter 5, concluding remarks and an outlook are given.

1.2 Background

1.2.1 Structural Features of Proteins

The sequence of residues in the polypeptide chain of a protein, also referred to as the primary structure, is determined by the DNA of the gene that encodes the protein. Chemical interactions between the residues themselves and the solvent fold the polypeptide chain into 3D structural conformations, which are held together by hydrogen bonds. These local structures are referred to as the secondary structure of a protein. The overall 3D structure is referred to as the tertiary structure and, if a protein consists of more than one polypeptide chain, the global structure of all

DSSP3	DSSP8	Name	Description
E	E	$\beta-$Sheet	Consecutive $\beta-$bridges
	B	$\beta-$Bridge	Parallel or anti-parallel
H	H	$\alpha-$Helix	4-Helix
	I	$\pi-$Helix	5-Helix
	G	$3_{10}-$Helix	3-Helix
C	S	Bend	Region with high curvature
	T	Turn	3-, 4- or 5-Turn
	-	None of the above	

TABLE 1.1: The DSSP8 and DSSP3 secondary structure states.

chains is called the quaternary structure. Anfinsen [13] discovered that the primary structure determines the secondary and tertiary structure directly, and that the latter in return determines the biological function of the protein, like, e.g. immune protection or transport of oxygen. This thesis concentrates on secondary structures. Pauling [14] identified in 1951 the two main types of secondary structures - the α-helix and the β-sheet. Hydrogen bonds between O and H residues form the core of secondary structures and the presence or absence of patterns of such bonds enables the identification of structural classes. The DSSP algorithm by Kabsch and Sander [15] determines the secondary structure by recognizing eight structural patterns in the atomic 3D coordinates (DSSP8). They are summarized in Table 1.1. The eight states can be further combined to three states (DSSP3), which are also shown in Table 1.1.

Moreover, certain features are closely related to the secondary structure. One is the relative solvent accessibility area (RSA). The RSA relies on the accessible surface area (ASA), i.e. the area of the protein which can get in direct contact with the solvent, usually given in units of square Ångströms ($\mathring{A}^2$). The RSA is the ASA relative to the surface area of the residue.[1] A protein behaves in a different fashion when being either buried by or exposed to the solvent [17]. Thus, the RSA relates closely to the structure. Also, thermal fluctuations constantly expose proteins to molecular vibrations, which results in structural flexibility. This is expressed by B-factors, i.e. atomic displacement factors, that resemble the degree of diffusion of the electrons around the equilibrium position due to thermal motion. It has been shown in [18] that the B-factors strongly depend on the corresponding RSA values. For example, an exposed residue with two buried neighbors has a lower mean B-factor than a buried residue with two exposed neighbors. This correlation makes the B-factor another interesting feature to take into consideration for the prediction of protein secondary structures.

1.2.2 Evolutionary Information

A key idea in protein secondary structure prediction is to include evolutionary information about the protein sequence alongside with the sequence itself. As suggested by

[1] In this thesis the normalization of [16] is used.

Rost and Sander [8] in 1993, this significantly pushed the Q3 accuracies of structure prediction methods to above 70%.

Two proteins are homologous when they share a common ancestor. Over time, evolution has created new proteins by randomly introducing mutations, which can be beneficial or detrimental for the fitness of the organism. If an amino acid has been replaced by another amino acid similar in size and with similar chemical properties, then this only has minor to none effects on the overall protein. This is called a conservative substitution. In contrast to that, radical substitutions have a major effect on the protein. Here the function is lost, which leads to the removal of the organism from the genetic pool. As a result, homologous proteins, which successfully have undergone evolution, share similar functions. This fact allows to transfer experimental annotations from one protein to another if both share a common ancestor, i.e. if they are homologs. Thus, evolution has a key function in structure prediction.

Modern alignment tools systematically align two or more amino acid sequences in order to identify concurring parts of the sequence. The result is a multiple sequence alignment (MSA) of related sequences. MMseqs2 [19] is the most recent and fastest alignment tool. However, it requires an enormous amount of memory. Moreover, BLAST [20] is a common iterative tool for searching databases to compute MSAs. A variant of BLAST, called PSI-BLAST [20], computes the position-specific scoring matrix (PSSMs) based on the sequences found in the current iteration and uses it for the next iteration during the search, whereas BLAST uses a pre-defined scoring matrix. The PSSM holds information on the frequencies of all substitutions per position. HHblits [21] is another tool to find high-quality MSAs. It is faster, finds up to two times more homologous proteins than PSI-BLAST and generates more accurate alignments. It relies on the pairwise alignment of profile hidden Markov models. In addition, position-specific gap penalties are included. This thesis focuses on MSAs which have been generated by HHblits and on PSI-BLAST PSSMs.

1.2.3 Machine Learning Techniques

Deep Neural Networks In the last decade, DNNs emerged as one of the most successful techniques in machine learning. Meanwhile, there exist various types of DNNs, which differ fundamentally in their structure and function. Inspired by adapting biological processes to machine learning, Convolutional Neural Networks (CNNs) mimic the behavior of the primary visual cortex of mammals that is responsible for processing visual input [22]. The basis of a CNN is formed by convolutional layers, which extract features by sliding a filter over the input. Due to the use of the same filter throughout the network, CNNs are translation- and shift-invariant. CNNs have become ubiquitous in computational solutions to visual recognition problems such as image classification, semantic segmentation and object detection [23].

Nevertheless, CNNs extend beyond visual information and have successfully been generalized to work on other data, e.g. audio signals [24], video signals [25] and graphs [26].

It has been shown that in CNNs shorter paths between the input and output result in a more accurate and more efficient learning of features. The DenseNet [27] model embraces this finding. While CNNs use the feature maps of the previous layer as the input for the current layer, they exhibit vanishing gradients when being very deep. DenseNet makes the feature maps of each layer accessible to each successive layer by concatenating the input of one convolutional layer to its output. This kind of information flow introduces better feature reuse and solves the vanishing gradient problem while requiring fewer parameters. In [27] it was shown that DenseNet models yield an improvement in performance over regular CNNs.

Another type of DNN is the recurrent neural network (RNN), which specifically operates on temporal sequences. Here, a single recurring weight matrix is used for the processing of sequences in each step. In that way the network can cope with time-dependent data. Backpropagation is done 'through time', leaving the network prone to vanishing gradients caused by the repeated derivation with respect to the recurrent weight matrix. To overcome this problem, Long Short-Term Memory networks (LSTMs) have been proposed by Schmidhuber and Hochreiter [28] in 1997. In contrast to RNNs, these networks make use of gates which control the information flow into and out of each cell, i.e. they steer what information is to be kept and what information is to be forgotten. As a result, long-term dependencies spanning over 1000 time steps can be learned. Moreover, LSTMs can be bidirectional (biLSTM) [29] with two recurrent layers, one working on the original sequence and the other one working on a reversed copy of it. This leads to the ability to not only learn from past time steps but also from future ones. LSTMs continue to play an increasing role in NLP, e.g. in sequence-to-sequence learning [30], speech recognition [31] and named entity recognition [32]. They have shown to be successful in other fields as well, e.g. in medical imaging [33].

Multi-Task Learning Instead of training one network per task independently, a network can be trained on multiple tasks simultaneously. This is called multi-task learning (MTL) [34]. Optimally, secondary tasks assist a primary task, which leads to an increased prediction performance. For that, the secondary tasks need to be sufficiently related to the main task. A MTL model takes one input and generates multiple outputs - one for each task. For example, HyperFace [35] jointly learns face detection, pose estimation, gender recognition and landmark localization in a MTL fashion. Applying MTL has shown to outperform single-task learning, see, e.g., [36]. MTL can either be done with hard or soft parameter sharing of the hidden layers. The soft parameter approach uses one model per task and regularizes between the different models in order to keep the parameters close to each other. In contrast, hard parameter shared MTL shares all hidden layers of one model among the tasks and only the final output layers are task-specific. The loss is computed by summing over the individual task-specific losses [34]. In this thesis we concentrate on hard parameter

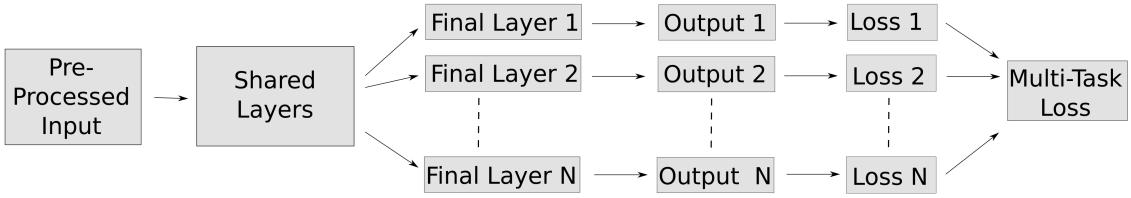


FIGURE 1.2: Overview of a multi-task learning model. N tasks are learned simultaneously and N outputs are produced. The N losses are combined to the total loss through summation.

shared MTL. Figure 1.2 gives an overview of a DNN model, which uses hard parameter shared multi-task learning to predict N targets simultaneously.

Word Embeddings Since data in NLP is often sparse and high-dimensional, methods which transform it into dense, low-dimensional data are ubiquitous. Together with the aim of finding similar representations for similar words, Word2Vec [37] offers a technique to obtain word embeddings with exactly these properties. For that, the SkipGram model [38] is used, which, for each word w_t in the input sentence, predicts the words occurring in a context window of size j to both sides of w_t. Furthermore, the model uses negative sampling: For each positive pair of (w_t, w_{t+i}), where w_{t+i} lies in the context window of w_t, k random negative pairs of words (w_t, n_i) are sampled, with word n_i outside the context window of w_t. The SkipGram model then learns to distinguish the positive and negative pairs of words [38]. As a result, the representation matrix of the words hold a real-valued vector entry for each word w_t. These vectors are used as the embeddings. Furthermore, the embeddings are able to deal with vector arithmetics, which show relations among the words and can help to solve analogy tasks. When dimensionality reduction techniques are applied, the embeddings can be displayed in a 2D space, where they are interpretable and where it can be observed that similar words are grouped together [37]. An advantage of Word2Vec is that the embeddings only need to be learned once and that they can hold valuable information on the data set they have been trained on. The embeddings can be reused in another machine learning task, transferring the learned information to another data set. This is called transfer learning.

1.2.4 ProtVec Word Embeddings

Adapting Word2Vec to the context of proteins, Asgari et al. [1] have applied Word2Vec to protein sequences (ProtVec) and gene sequences (GeneVec). The ProtVec embeddings have been trained on 546 790 proteins from the SwissProt database [39] by shifting a window of size three over the sequence and splitting each protein into overlapping 'words' of three residues. A context size of 25 residues was used. For each 3-gram, ProtVec obtains a dense embedding vector of length 100 with values in the range from

0 to 1. It was shown that the learned embeddings hold valuable biophysical and biochemical properties, like polarity or hydrophobicity, by clustering them in a 2D space. In [1] it was further suggested that deep learning tasks for protein structure prediction might benefit from the use of ProtVec embeddings.

1.3 Related Work

Deep learning has a rich history in protein secondary structure prediction [40]–[42]. A landmark success of Q3 accuracy above 70% was reached by Rost and Sander [8] in 1993 by including evolutionary information into the input of a multi-layered neural network. Today, up to 85% Q3 accuracy is reached by building on this idea, compare [10]–[12].

Moreover, in various works it was proposed to predict related features simultaneously. The recently suggested model SPIDER3 [43] uses a biLSTM to capture long-range dependencies in the amino acid sequence and thus achieves 84.5% Q3 accuracy. SPIDER3 uses an iterative multi-step algorithm to predict ASA, backbone torsion angles and contact numbers. A combination of the PSI-BLAST PSSM matrix, the HHblits MSA and seven physio-chemical properties is taken as the input. Li and Yu [9] predict DSSP states in combination with solvent accessibility through hard parameter shared MTL. To this end, a CNN combined with a bidirectional RNN is trained. Their model achieves 84.0% Q3 accuracy. The One-hot encoded representation concatenated with the rescaled PSI-BLAST PSSM matrix serves as the input here. NetSurfP2 [12] predicts DSSP3 and DSSP8 structures, backbone angles, ASA and RSA, as well as structural disorder with a combination of a CNN and a two-layered biLSTM. Hard parameter shared MTL is again applied here. The authors use the HMM profile generated by HH-Suite as well as the One-hot encoded sequence for the input. This model reaches an average Q3 accuracy of 85% which is the highest achieved accuracy for DSSP3 structure prediction to this day.

As predicting DSSP3 structures is getting more and more elaborate, DSSP8 prediction is explored as well with continuously improving results. Zhang et al. [18] achieve a Q8 accuracy of 74% by combining a CNN, a residual network and a bidirectional RNN with ensemble learning. Wang et al. [44] combine conditional random fields with neural networks and achieve a Q8 accuracy of 72% while the PSI-BLAST PSSM matrix serves as the input. The above mentioned SPIDER3 and NetSurfP2 reach a Q8 accuracy of 73% and 72%-75% on average, respectively.

It is important to note that these results can not be compared directly, because different data sets are used and - in DSSP3 classification - the mappings from DSSP8 to DSSP3 structures differ. Finally, to the best of our knowledge, there is no published work using ProtVec embeddings for secondary structure prediction yet.

Chapter 2

Implementation

In the following, our work flow is presented. As a first step, we analyze the given data set. Next, we describe the implementation of our structure prediction model in detail. The first part of the model handles data pre-processing, the second part consists of the DNN architectures and the third part employs the multi-task learning. These three parts are combined to form the overall model which is then trained, tested and evaluated. Furthermore, the training and testing procedure and the chosen hyperparameters are given. The implementation is realized in PyTorch.[1]

2.1 Data Analysis

Data set The data set consists of 2935 protein sequences and their according structural information, that is the DSSP3 structures, the DSSP8 structures, the RSA values and the B-factors.[2] The proteins have been taken from the PDB database, where only proteins with a resolution higher than 2.5$\AA$ have been considered in order to guarantee a sufficiently high quality. For each protein, a MSA has been computed with the help of HHblits [21].[3] Besides the three and eight structure states (see Table 1.1), the resulting sequences also hold two auxiliary states X and Y. Here, X represents a residue position of missing coordinates in the original PDB files. This indicates that no measurements could be made, since, e.g. in X-ray crystallography, flexible regions and hydrogen atoms are not being captured [2]. Y corresponds to a position where the DSSP algorithm was not able to clearly decide on one state but rather determined different states for the multiple protein chains. Altogether, the secondary structure targets thus have five (DSSP3) and ten (DSSP8) states. The RSA targets are normalized PDB RSA values. Here, the normalization suggested in [16] is used. The values lie in the range from 0 to 1 and the more a value approaches one, the more exposed its associated position is. The B-factors have been taken from the B-factor DataBank [45], a normalized and consistent version of the PDB B-factors. They lie in the range from -3 to 8 and represent the Z-score. This is a measurement which

[1] Python3 and PyTorch v1.0.1 are used.

[2] The data set, including the targets, has kindly been provided by Michael Heinzinger.

[3] First HHblits is called with the parameters -maxfilt 100000 -realign_max 100000 -B 100000 -Z 100000 -n 5 -e 0.1 -all hhfilter -id 90 -neff 15 -qsc -30. The alignments are then filtered with HHfilter using parameters -id 90 -neff 15 -qsc -30.

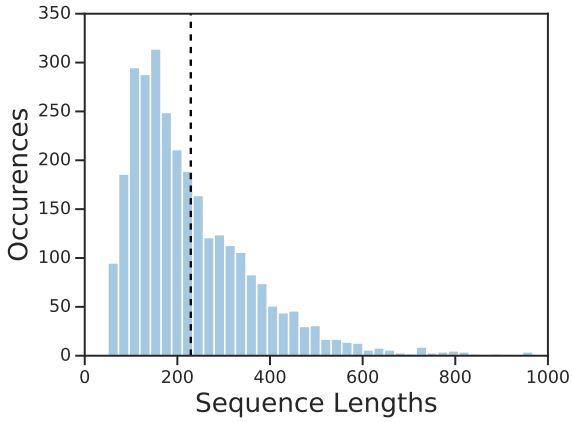


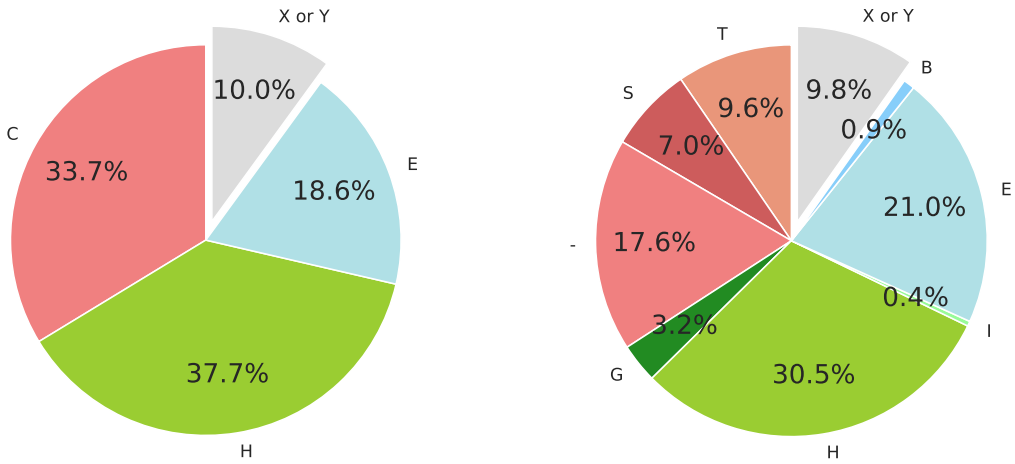
FIGURE 2.1: Distribution of protein lengths in the data set. The average length of the 229 residues is indicated by the dotted line.

expresses the number of standard deviations from the mean, which is 0. The higher the value is, the more flexible the associated position is.

Data Analysis In machine learning, data analysis is the first and crucial step to understand the data in depth and to be able to adjust the learning algorithm to the data set. On average, the protein sequences in our data set have a length of 229 residues while the lengths range from 50 to 968 residues. Figure 2.1 shows the distribution of sequence lengths in the data set.

Analyzing the class distribution in the data set is fundamental to recognize inherent imbalances. While all of the DSSP3 states occur quite often (State C with 33.7%, state H with 37.7% and state E with 18.6% of all residues), the DSSP8 states have an unequal distribution (states H (30.5%), E (21%) and state − (17.6%) occur most often, followed by states T (9.6%), S (7%), G (3.2%), B (0.9%) and state I (0.4%)). The X and Y states make up 10% and 9.8% for DSSP3 and DSSP8, respectively. This means that around every tenth residue in the data set holds no relevant information for the classification. Figure 2.2 shows the class distributions graphically.

How often one secondary structure state occurs in a row on average naturally differs for each state. Here, L_c refers to the length of a sub-sequence of amino acids only being assigned to state c. For each of the three DSSP3 states, occurrences of sub-sequences of length L_c are counted in the whole data set. From that, we found that, on average, instances of state C occur four times, instances of state H occur ten times and instances of state E occur five times in a row. L_C and L_E are mostly smaller than 30 and 20, respectively, and L_H up to 70 do occur. In DSSP8, on average, instances of state H occur 11 times, instances of state E occur six times, instances of state I occur five times and instances of state G occur three times in a row, while all other states occur only one or two times in a row. Figure 2.3 displays the distribution



(A) DSSP3 (B) DSSP8

FIGURE 2.2: Distribution of the DSSP states in the data set. The left side shows the DSSP3 classification and the right side shows the DSSP8 classification. The state-abbreviations are as given in Table 1.1. State X and Y are auxiliary states of positions where no measurements could be made and where the DSSP algorithm determined different states in the different sequence chains, respectively.

of counts of L_C, L_H, L_E for the DSSP3 states in the whole data set. Also, the DSSP3 and DSSP8 structure states share an average distance of five residues between two $\beta-$sheets.

Furthermore, it is helpful to investigate the RSA and B-factor targets. The results are given in Figure 2.4. Roughly 85% of the residues are buried and 15% are exposed, given that the classification threshold is 0.5. The average RSA values per sequence lie between 0.2 and 0.3 for the majority of the sequences. Due to the B-factors representing the Z-score, their mean is 0. About 12% of all residues are considered not flexible ($x \leq -1$), 75% average-flexible ($-1 < x < 1$) and 13% flexible ($x \geq 1$).

2.2 The Overall Workflow

In the following our workflow of the prediction model implementation is described. First, the three parts of our model are presented, i.e. the data pre-processing, the network architectures and the multi-task learning. Then, it is explained how these parts form the overall model.

2.2.1 Data Pre-Processing

The data is pre-processed in five different ways. It is important to note that this is done per residue to encode the input data in the most detailed way possible. Here, L

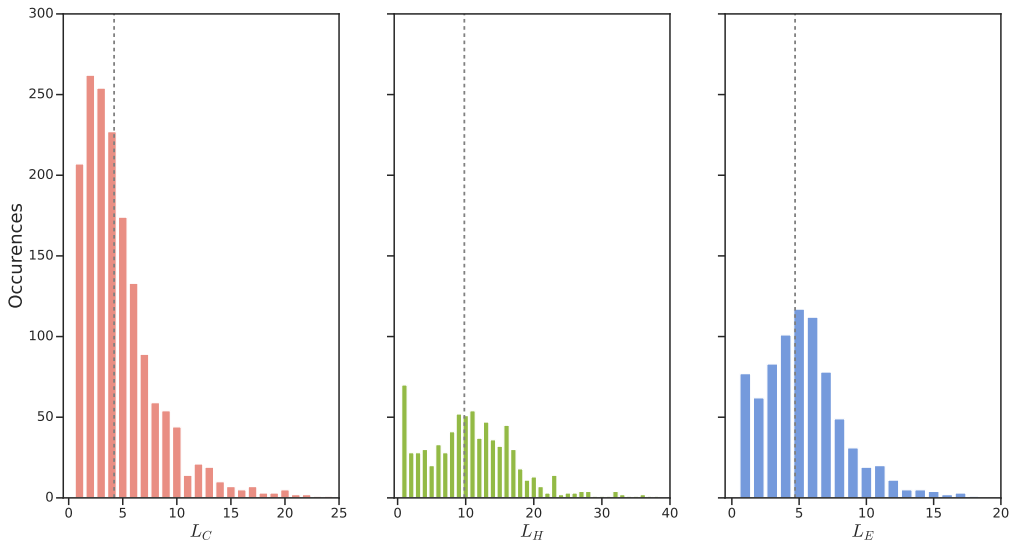
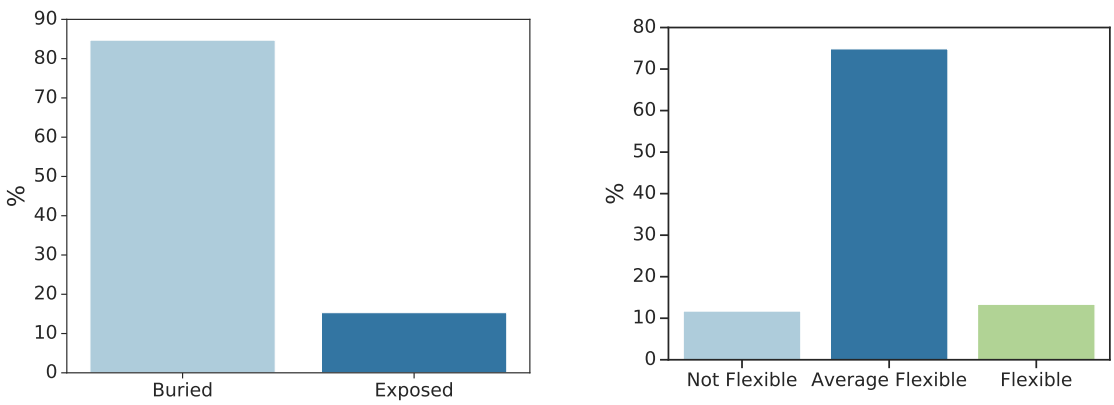


FIGURE 2.3: Distribution of lengths of sub-sequences with only one structure state. L_c refers to the length of a sub-sequence consisting solely of state c, given in numbers of positions. The x-axis shows the lengths and the y-axis shows the number of occurrences in the whole data set. The average lengths are indicated by the dashed lines. Outliers are cut off for visual reasons. These are L_H of lengths 42, 46, 47 and 60 as well as L_C of lengths 27, 28 and 30 and a L_E of length 22.



(A) RSA values (B) B-factors

FIGURE 2.4: Distribution of the RSA values and B-Factors in the data set. For the RSA values, $x < 0.5$ is considered to be buried and $x > 0.5$ is considered to be exposed. For the B-factors, $x < -1$ is considered not flexible, $-1 < x < 1$ is considered average flexible and $x > 1$ is considered flexible.

represents the length of the original sequence. The five resulting input representations are:

- **One-hot** (One-hot encoding without evolutionary information): The amino acid sequence is encoded as a One-hot matrix $\in \mathbb{R}^{L\times 20}$. The rows in the matrix correspond to the positions in the sequence. The columns represent the 20 amino acids. For each position, the entry of the respective amino acid in the row is set to one, while all other entries in that row are set to zero.

- **ProtVec** (ProtVec vectors without evolutionary information): A sliding window of size three is applied to the amino acid sequence and each residue and its two neighbors are encoded by the corresponding ProtVec vector of size 100. Since the first and last position have only neighbors to one side, and thus no corresponding ProtVec vector is available, the resulting matrix is zero-padded in the first and last row. The result is a matrix $\in \mathbb{R}^{L\times 100}$.

- **ProtVec (MSA)** (ProtVec vectors with evolutionary information): To include evolutionary information, the ProtVec representation like the one described above is computed for each alignment in the protein's MSA. In order to obtain the final representation, the resulting sequence of ProtVec matrices then is averaged. This again results in a matrix $\in \mathbb{R}^{L\times 100}$.

- **PSSM** (PSSM matrix with evolutionary information): The PSSM matrix is computed with the PSI-BLAST algorithm for the given MSA of the protein. This results in a matrix $\in \mathbb{R}^{L\times 20}$.

- **PSSM+ProtVec** (ProtVec representation with evolutionary information combined with PSSM matrix): The ProtVec representation matrix of the MSA is concatenated with the PSSM matrix. This results in a matrix $\in \mathbb{R}^{L\times 120}$.

The longest sequence in the data set is 968 residues long. To standardize the input size, the representations of all shorter sequences are zero-padded at the end of the sequences. Thus, $L = 968$ for all protein sequences after pre-processing.

To provide a numerical representation of the secondary structure targets, the mapping shown in Table 2.1 is used.

DSSP3		DSSP8	
C → 0	X → 2	H → 0	G → 5
H → 1	Y → 2	E → 1	B → 6
E → 2		I → 2	− → 7
		S → 3	X → 7
		T → 4	Y → 7

Table 2.1: Mapping to represent the DSSP structures numerically.

During training and testing, classes X and Y are masked out in such a way that they do not influence the learning and prediction process. This is done by creating a mask vector $\in \{0,1\}^L$ for the DSSP3 targets and the DSSP8 targets each. It contains zeros where the positions of the X and Y states and the zero-padded positions are situated and is otherwise set to one. Two further masks are created in the same way for the RSA values and B-factors. Here, NaN values and zero-padded positions are masked out. These masks are then used in the training and testing process.

Altogether, after this pre-processing step, there are five different input representations, four target sequences and four masks available for each protein.

2.2.2 Network Architectures

In the following, the details of our six different implemented network architectures are given. We employ one CNN, two LSTMs and one combination of those two network types. Furthermore, we apply the idea behind DenseNet layers to one CNN and one CNN-BiLSTM network. For simplicity, the architectures are described with respect to the ProtVec representation as the input and DSSP3 structure prediction only, i.e. no multi-task learning is considered and the final layers are DSSP3 structure-specific. The architectures are chosen in such a way that they possess about the same number of learnable parameters and thus about the same capacity. This is necessary to be able to compare different networks in a meaningful manner. However, the DenseNet inspired layers of the DenseCNN, for example, can then not develop their full potential over the normal CNN layers, because the sizes of the feature maps were reduced to keep the amount of parameters equal.

CNN Our CNN architecture consists of three one-dimensional convolutional layers. This allows the network to convolve also over regions with impossible combinations of predictions like e.g only two H states in a row, even though helices with a length smaller than three do not exist. The employed network structure can be seen in Figure 2.5. The first layer has a filter of size one and maps the input to 75 features.

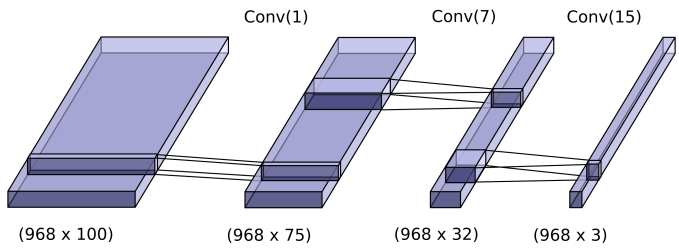


FIGURE 2.5: CNN architecture. The data size is shown as (*length* × *width*). The filters map one input position to the corresponding output position. The filter size is given in the convolution label.

The second layer has a filter of size seven, which reflects that for each residue a context window of size three to both sides is taken into account, and maps the input to 32 features. The third layer has a filter of size 15, which reflects that a context window of seven residues to both sides is taken into account, and maps to three features. In order to obtain a prediction of the same length as the input, we do not use pooling layers. Also, instead of a final fully connected layer, the logarithmic Softmax function is directly applied to the output. As activation function a rectified linear unit (ReLU) is used after every convolutional layer, as it yielded slightly better results than a scaled exponential linear unit (SeLU).

CNN with DenseNet-inspired layers (DenseCNN) In order to make use of the advantages coming with DenseNet, we implemented a CNN with DenseNet-inspired layers. To this end, the output of each convolutional layer is concatenated with the input of the same layer, so that both, the information before and after the feature extraction, is available to the subsequent layers. Our employed network consists of three layers. The kernels are of sizes one, seven and 15 and the output feature sizes are 32, 22 and three, respectively. The network structure is shown in Figure 2.6.

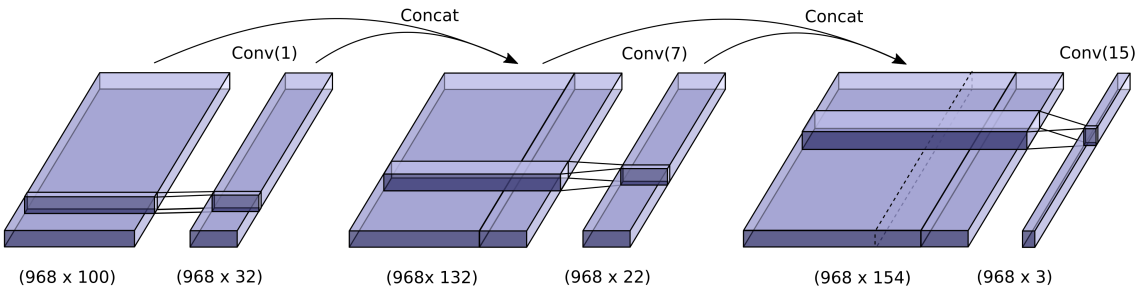


FIGURE 2.6: DenseCNN architecture. The data size is shown as $(length \times width)$. The filters map one input position to the corresponding output position. The filter size is given in the convolution label.

LSTM Our LSTM network is two-layered and has hidden states of feature size 34. The fact that the network is two-layered means that two LSTMs are stacked and the second one takes the output of the first one as input to compute the final results. We use bias weights and to map the LSTM output to the target space, a fully-connected linear layer is used.

BiLSTM To evaluate the effect of making the network bidirectional, we also implemented the above LSTM as a biLSTM.

CNN-BiLSTM (Hybrid) We furthermore implemented a combination of a CNN and a biLSTM. In this hybrid architecture, the CNN is responsible for the feature extraction of the inputs while the biLSTM operates on the output of the CNN and

deals with the prediction of the targets. The biLSTM output is then the final output. The CNN has two convolutional layers with kernel sizes one and seven, mapping their input to 42 and 50 features, respectively. The biLSTM has one layer with hidden states of size 34 and a fully-connected linear layer for the final output. Again, the ReLU activation function is applied after each convolutional layer.

CNN-BiLSTM with DenseNet inspired layers (DenseHybrid) Finally, we implemented a hybrid architecture like the previous one while now making use of DenseNet-inspired layers. The CNN in this architecture has three convolutional layers with kernels of sizes one and seven, mapping to eight and 30 features, respectively. The biLSTM has one layer and hidden states with feature size six. The activation function and the final layer are as in the Hybrid architecture above.

2.2.3 Multi-Task Learning

In this thesis, we used hard parameter shared multi-task learning to predict secondary structures together with RSA values and B-factors. The structures are represented through discrete classes whereas the latter two are represented by real values. Therefore, classification is performed for the secondary structures and linear regression for the secondary tasks. This is realized by task-specific final layers. For the classification, a fully-connected layer, which maps to three or eight features, is applied in combination with the log-softmax function. For each of the two linear regression tasks, a fully connected layer is applied, which maps to one feature. For the RSA prediction task we use ReLU as the activation function and since the B-factors can be negative we use the TanhShrink activation function here.

Regarding multi-task learning, four different variants are implemented:

- Not using multi-task learning and only predicting the secondary structures (either DSSP3 or DSSP8)
- Using multi-task learning to predict the secondary structures (either DSSP3 or DSSP8) and the RSA values simultaneously
- Using multi-task learning to predict the secondary structures (either DSSP3 or DSSP8), the RSA values and the B-factors simultaneously
- Using multi-task learning to predict DSSP3 and DSSP8 secondary structures, the RSA values and the B-factors simultaneously.

2.2.4 Overall Prediction Model

Altogether, we now have at our disposal: Five different input representation types, six different DNN architectures and three multi-task learning variants (and additionally the option to not use multi-task learning at all). These three parts are

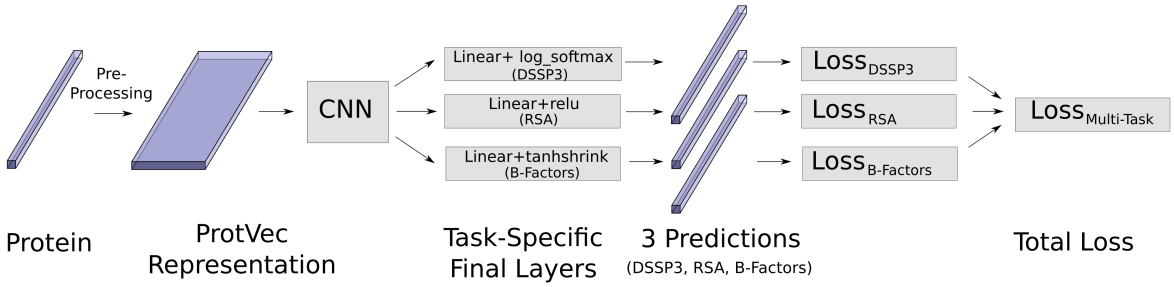


FIGURE 2.7: One sample configuration of our overall model. In this example, the ProtVec representation is the input of the network. Then, the CNN architecture and multi-task learning of DSSP3 structures, RSA-values and B-factors are employed.

assembled to give the final setup for our secondary structure prediction model. As a result, $5 \times 6 \times 4 = 120$ configurations of our model would be possible. Of course, training, testing and evaluating all of them is beyond the scope of this thesis. Consequently, an educated selection of configurations is used, for details see Chapter 3.

One sample configuration of our setup is shown in Figure 2.7 with the ProtVec representation input, the CNN architecture and multi-task learning of DSSP3 structures, RSA values and B-factors.

2.3 Training Parameters and Training Procedure

We have implemented our overall model for secondary structure prediction as described above. Next, the model needs to be trained. To this end, as usually done in machine learning, the data set is split into a training set, a validation set and a test set. The employed splitting is about 20% of all samples for the test set, about 20% of the remaining samples for the validation set. The rest is used for the test set. To be precise, the training set holds 1861 proteins, the validation set holds 476 proteins and the test set holds 598 proteins.

For the structure prediction, a well-known variant of the Adam-optimizer [46], as suggested in [47], is used. Plain structure prediction is a classification task and thus, we use the negative log likelihood loss. For the regression tasks of predicting the RSA values and B-factors, we use the mean squared error loss. In all three tasks, the unreduced loss $\in \mathbb{R}^L$ is element-wise multiplied with the corresponding mask, so that the loss at irrelevant positions is ignored. The entries of the loss-vector are summed and then are averaged over the amount of relevant positions. This way, the average loss is finally obtained for each task.

As shown in Section 2.1, the DSSP3 states C and H are observed more often than state E. To compensate for this, class balancing is used. To this end, the inverse percentage of each state serves as weighting. Then, for example, the weight for state E, which holds 18.6% of all positions, is $\frac{1}{0.186} = 5.376$. To include these weights, the structure mask of each protein is multiplied with the weight that corresponds to the structure at the individual positions. The positions which have been masked out before, thus remain zero. However, for the DSSP8 classification, no weighting is employed since further experiments showed no improvement with weights in that case. A constant dropout rate of 0.3 is used. The learning rates for the individual networks are summarized in Table 2.2.

CNN	DenseCNN	LSTM	biLSTM	Hybrid	DenseHybrid	DenseHybrid8
0.0001	0.0001	0.005	0.001	0.0001	0.001	0.005

Table 2.2: Learning rates for the individual networks. DenseHybrid8 refers to the DenseHybrid architecture when learning DSSP8 structures, all other learning rates refer to DSSP3 structure prediction.

Chapter 3

Experimental Evaluation

In this chapter, we first state the metrics used to evaluate the performance of the different DNNs. Then, we give the results of our experiments on different model configurations. In the first experiment, we investigate the behavior of the CNN for different input representations. In the second experiment, we compare the performance of the different network architectures. We examine the effect of multi-task learning in a third experiment. In a fourth and fifth experiment, we study the learning of DSSP8 structures and in the sixth experiment, we finally compare the model configuration with the best performance so far for all different input representations.

3.1 Performance Metrics

To evaluate how a neural network performs, performance metrics have to be chosen. A classification task has discrete categorical classes to predict from. This means that the amount of correctly and incorrectly classified labels can be counted. There are four options for each prediction. It can either be

- True Positive (TP): Predicted correctly to be in class c,
- True Negative (TN): Predicted correctly to not be in class c,
- False Positive (FP): Predicted incorrectly to be in class c or
- False Negative (FN): Predicted incorrectly to not be in class c,

where c denotes a specific class to be considered.

A confusion matrix is a visually appealing way to display these four cases for all classes at the same time, where the rows represent the true labels and the columns the predicted labels. Accuracy measures the ratio of correctly classified labels to all labels, i.e.

$$\mathrm{Acc} = \frac{\mathrm{TP}+\mathrm{TN}}{\mathrm{TP}+\mathrm{FP}+\mathrm{FN}+\mathrm{FP}}. \tag{3.1}$$

However, sometimes measuring the accuracy is not sufficient. One way to evaluate performance in more detail and with respect to each class is to use precision and

recall. Precision resembles the ratio of correctly predicted observations of class c to all predicted observations of class c. Recall resembles the ratio of correct predictions of class c to all true observations of class c. Thus,

$$\mathrm{Prec}_c = \frac{\mathrm{TP}_c}{\mathrm{TP}_c + \mathrm{FP}_c} \qquad \mathrm{Rec}_c = \frac{\mathrm{TP}_c}{\mathrm{TP}_c + \mathrm{FN}_c}. \tag{3.2}$$

The F-measure

$$\mathrm{F}_c = 2 * \frac{\mathrm{Prec}_c * \mathrm{Rec}_c}{\mathrm{Prec}_c + \mathrm{Rec}_c} \tag{3.3}$$

summarizes the precision and recall of a classifier. It resembles the harmonic mean of precision and recall [48]. In the following, these measures are first computed for each protein in the test set and then the results are averaged over all 598 proteins. Moreover, besides the mean, the standard error is calculated in this way.

3.2 Results

In the following, our experimental results will be presented. We evaluated all network architectures after 100 training epochs. This was sufficient to reach the saturation of the performance. We focus on accuracy rather than on loss because accuracy gives a better understanding of the performance. First, the CNN, which is considered to be the simplest network, is evaluated on different inputs to investigate the effect of using ProtVec embeddings. Then, the best performing ProtVec representation is evaluated on all other network architectures. After that, the effect of multi-task learning is studied. Finally, the so far best model is evaluated on DSSP8 structures and also on all input representations. Moreover, we examine if learning DSSP8 structures and mapping the predictions to DSSP3 structures holds an improvement over the direct learning of DSSP3 structures.

3.2.1 Effect of ProtVec for the CNN Architecture

In our first experiment, the effect of using different input representations is evaluated on the CNN architecture. This gives a direct comparison between the use of ProtVec embeddings and not using them. The learning progress can be seen in Figure 3.1. The detailed results of the inputs, which do not include evolutionary information, are given below, followed by the detailed results for the inputs which do include evolutionary information.

No Evolutionary Information First, we do not consider evolutionary information and use the One-hot and the ProtVec representation of the single sequences. The learning process can be seen in Figure 3.1 (orange and dashed blue lines). We can observe that the performance is similar for both inputs during the first 50 epochs. Then, the One-hot input outperforms the ProtVec input. After 100 epochs, the first

one reaches 64.45% accuracy and the latter 66.97%. This is a difference of 2.52%. The resulting confusion matrices for both input types on the test set can be seen in Figure 3.2.

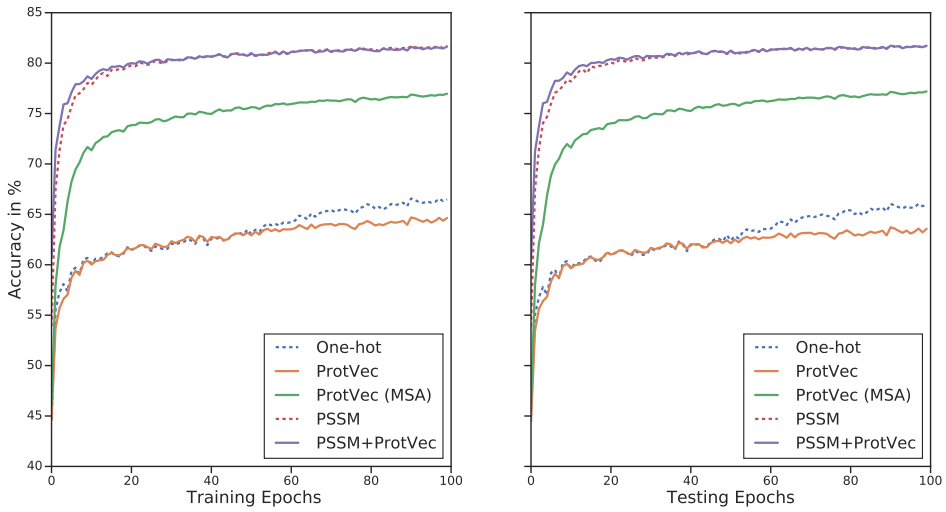
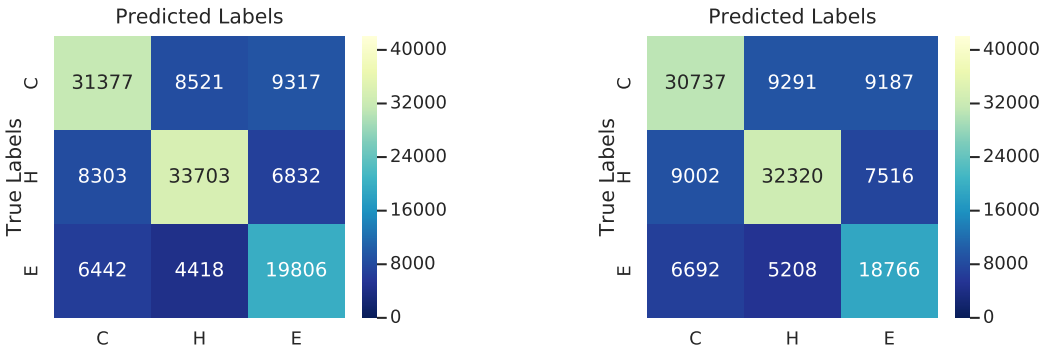


FIGURE 3.1: Learning curves of training (left) and testing (right) for the CNN described in Section 2.2.2 with the different input representations, as described in Section 2.2.1, for 100 epochs. DSSP3 structures are learned and no MTL is used.



(A) One-hot (B) ProtVec

FIGURE 3.2: Confusion matrices for the CNN network described in Section 2.2.2 after 100 epochs without taking evolutionary information into account. The left side shows the confusion matrix for the One-hot encoded input, the right side for the ProtVec representation as described in Section 2.2.1. The values state the absolute counts of structure states over the whole test set.

Table 3.1 summarizes the accuracy and the different class-specific performance measurement results. Note here that for Figure 3.2 the whole test set is evaluated, whereas in Table 3.1 the measurement results are evaluated per protein of the test set, just as described in Section 3.1. It becomes clear that the class-specific measurement results are higher without exception for the One-hot encoded input.

Input	$Prec_C$	$Prec_H$	$Prec_E$
One-hot	$\mathbf{67.74 \pm 0.5}$	$\mathbf{65.74 \pm 1.0}$	$\mathbf{50.04 \pm 1.1}$
ProtVec	65.60 ± 0.5	62.68 ± 1.0	48.69 ± 1.0

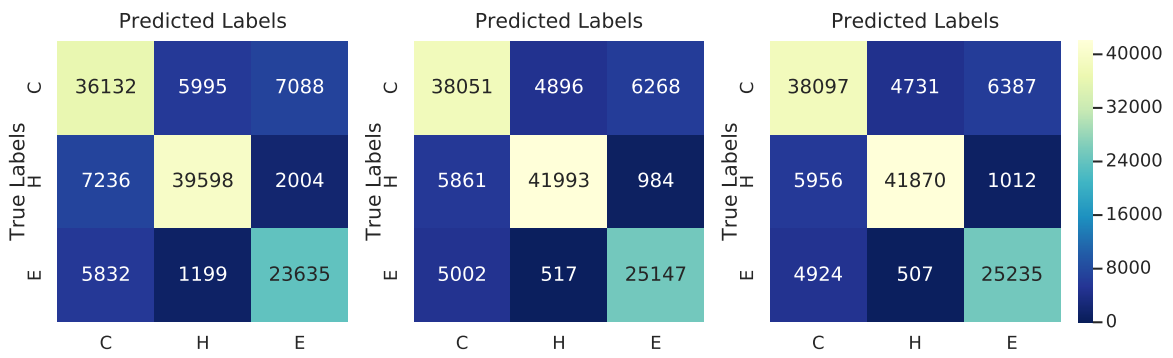
Input	Rec_C	Rec_H	Rec_E
One-hot	$\mathbf{64.17 \pm 0.5}$	$\mathbf{66.10 \pm 0.9}$	$\mathbf{63.62 \pm 0.7}$
ProtVec	62.50 ± 0.5	62.92 ± 0.9	60.17 ± 0.8

Input	F-measure$_C$	F-measure$_H$	F-measure$_E$	Acc
One-hot	$\mathbf{65.07 \pm 0.4}$	$\mathbf{67.05 \pm 0.7}$	$\mathbf{58.89 \pm 0.7}$	$\mathbf{66.97 \pm 0.3}$
ProtVec	63.14 ± 0.4	64.08 ± 0.7	56.04 ± 0.7	64.45 ± 0.3

TABLE 3.1: Class-specific measurement results and accuracy averaged over the proteins in the test set after 100 epochs for the CNN described in Section 2.2.2. Evolutionary information is not taken into account. DSSP3 structures are learned and no MTL is used. The input representations are described in Section 2.2.1. The best values per column are highlighted.

Including Evolutionary Information Then, we evaluate the usage of ProtVec embeddings when taking evolutionary information into account. The raw PSSM matrix as well as the ProtVec representation of the MSA and also their combination is used. The learning progress can again be seen in Figure 3.1 (green, purple and dotted red lines). It can be observed that the performance in the first 20 epochs is best for the PSSM+ProtVec input. After that the PSSM input catches up. Both reach about the same accuracy of 82.45% and 82.38% after 100 epochs, respectively. The performance for the ProtVec encoded MSA is 77.91% accuracy. The resulting confusion matrices for all three input types on the test set can be seen in Figure 3.3 and Table 3.2 summarizes the class-specific results.

Altogether, we observe a substantial increase of $\geq 11\%$ when evolutionary information is included. This confirms the importance of evolutionary information in secondary structure prediction. Moreover, the CNN performs best for the PSSM+ProtVec input and for the raw PSSM matrix.



(A) ProtVec (MSA) (B) PSSM (C) PSSM+ProtVec

FIGURE 3.3: Confusion matrices for the CNN described in Section 2.2.2 after 100 epochs while taking evolutionary information into account. The inputs are the ProtVec (MSA) (left), PSSM (center), PSSM+ProtVec (right), described in Section 2.2.1. The values state the absolute counts of structure states over the whole test set.

Input	$Prec_C$	$Prec_H$	$Prec_E$
MSA ProtVec	73.42 ± 0.4	80.12 ± 0.8	65.77 ± 1.1
PSSM	78.37 ± 0.4	86.37 ± 0.6	$\mathbf{72.23 \pm 1.0}$
PSSM+ProtVec	$\mathbf{78.62 \pm 0.4}$	$\mathbf{86.62 \pm 0.6}$	71.40 ± 1.0

Input	Rec_C	Rec_H	Rec_E
MSA ProtVec	73.05 ± 0.4	74.52 ± 1.0	73.42 ± 0.9
PSSM	$\mathbf{76.89 \pm 0.4}$	$\mathbf{79.96 \pm 0.9}$	78.74 ± 0.9
PSSM+ProtVec	76.54 ± 0.5	79.10 ± 1.0	$\mathbf{79.12 \pm 0.9}$

Input	F-measure$_C$	F-measure$_H$	F-measure$_E$	Acc
MSA ProtVec	72.74 ± 0.3	80.01 ± 0.6	73.66 ± 0.6	77.91 ± 0.3
PSSM	$\mathbf{76.99 \pm 0.3}$	84.45 ± 0.5	78.36 ± 0.6	$\mathbf{82.45 \pm 0.3}$
PSSM+ProtVec	76.94 ± 0.3	$\mathbf{84.88 \pm 0.5}$	$\mathbf{78.64 \pm 0.5}$	82.38 ± 0.3

TABLE 3.2: Class-specific measurement results and accuracy averaged over the proteins in the test set after 100 epochs for the CNN described in Section 2.2.2. Evolutionary information is taken into account. DSSP3 structures are learned and no MTL is used. The input representations are described in Section 2.2.1. The best values per column are highlighted.

3.2.2 Performance of the Different Network Architectures

Next, we evaluate the performance of the different network architectures. This is done with the ProtVec representation for which the CNN performed best in the preceding experiment, which is the PSSM matrix in combination with the ProtVec encoding of the MSA. The performance can be seen in Figure 3.4. It becomes clear that all networks reach a Q3 accuracy of $82 - 83\%$ after 100 epochs, except for the LSTM architecture, which only reaches 78.28%. The class-specifc results are given in Table 3.3.

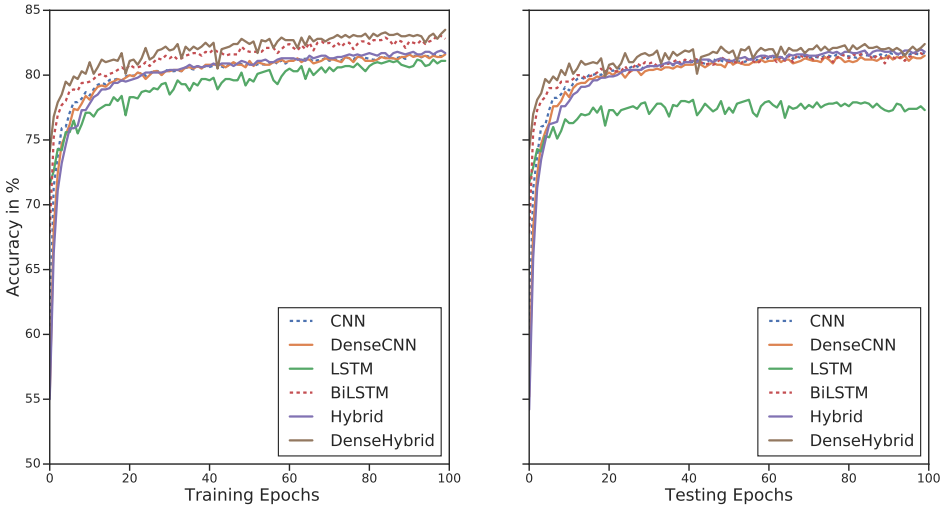


FIGURE 3.4: Learning curves of training (left) and testing (right) for 100 epochs for all networks described in 2.2.2. The PSSM+ProtVec input described in Section 2.2.1 is used. DSSP3 structures are learned and no MTL is employed.

Input	$Prec_C$	$Prec_H$	$Prec_E$
LSTM	74.72 ± 0.5	79.21 ± 0.8	64.64 ± 1.1
biLSTM	78.75 ± 0.4	85.05 ± 0.6	75.25 ± 0.9
DenseCNN	77.44 ± 0.4	86.16 ± 0.7	72.36 ± 1.0
Hybrid	$\mathbf{78.92 \pm 0.4}$	$\mathbf{87.30 \pm 0.5}$	74.35 ± 0.9
DenseHybrid	78.74 ± 0.4	86.38 ± 0.6	$\mathbf{76.46 \pm 0.9}$

Input	Rec_C	Rec_H	Rec_E
LSTM	71.69 ± 0.5	74.72 ± 0.9	77.57 ± 0.9
biLSTM	75.96 ± 0.5	81.38 ± 0.9	77.40 ± 1.0
DenseCNN	77.43 ± 0.5	78.27 ± 1.0	77.94 ± 0.9
Hybrid	75.84 ± 0.5	79.34 ± 1.0	$\mathbf{78.84 \pm 0.9}$
DenseHybrid	$\mathbf{78.13 \pm 0.4}$	$\mathbf{81.89 \pm 0.9}$	77.70 ± 0.9

Input	F-measure$_C$	F-measure$_H$	F-measure$_E$	Acc
LSTM	72.57 ± 0.4	78.30 ± 0.7	74.86 ± 0.7	78.28 ± 0.3
biLSTM	76.81 ± 0.4	83.96 ± 0.5	78.92 ± 0.5	82.27 ± 0.3
DenseCNN	76.94 ± 0.4	84.29 ± 0.5	78.77 ± 0.5	82.22 ± 0.3
Hybrid	76.70 ± 0.4	$\mathbf{85.35 \pm 0.5}$	78.95 ± 0.5	82.53 ± 0.3
DenseHybrid	$\mathbf{77.75 \pm 0.3}$	84.99 ± 0.5	$\mathbf{79.52 \pm 0.5}$	$\mathbf{83.17 \pm 0.3}$

TABLE 3.3: Class-specific measurement results and accuracy averaged over the proteins in the test set after 100 epochs for the networks described in Section 2.2.2. The PSSM+ProtVec input described in 2.2.1 is used. DSSP3 structures are learned and no MTL is employed. The best values per column are highlighted.

Altogether, the best performing architecture is the DenseHybrid network with 83.17% Q3 accuracy, which is 0.72% better than the performance of the CNN architecture on the same input representation.

3.2.3 Effect of Multi-Task Learning

So far, we have only predicted the secondary structure. Now, we will investigate how additional predictions of the RSA values and B-factors affect the performance. To this end, we use the DenseHybrid network and the PSSM+ProtVec input representation, which yielded the best results in the preceding experiment. As described in Chapter 2, three variants of multi-task learning have been implemented. These are compared in the following.

Figure 3.5 shows the development of the performance for the three different multi-task variants and Table 3.4 summarizes the class-specific measurement results. It can be observed that with multi-task learning the DenseHybrid network reaches an accuracy of roughly 83%, independent of the choice of secondary tasks. The inclusion of the B-Factors results in a marginal decrease of 0.38% compared to only learning the RSA values as a secondary task. When learning multiple targets in this way, the model outputs additional predictions which we can also have a look at. Figure 3.6 shows the additional predictions of RSA values and B-factors for a randomly chosen protein from the test set. We can observe that the locations of the peaks and valleys of the RSA values and also the B-factors are predicted well.

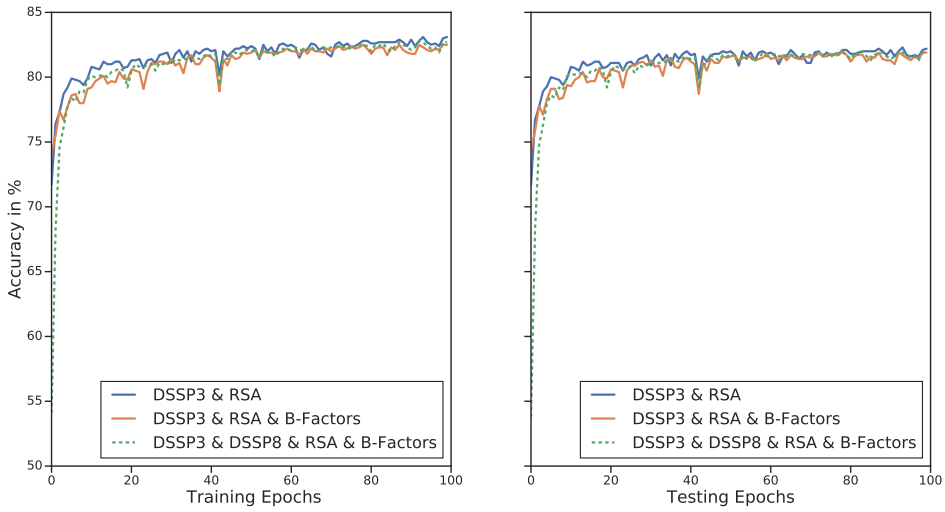


FIGURE 3.5: Learning curves of training (left) and testing (right) for the DenseHybrid network of Section 2.2.2 for 100 epochs with the different variants of MTL of Section 2.2.3. The PSSM+ProtVec input described in Section 2.2.1 is used. The legend shows the learning tasks.

Input	$Prec_C$	$Prec_H$	$Prec_E$
Multi 2	**78.06 ± 0.4**	86.66 ± 0.6	**76.44 ± 0.9**
Multi 3	77.48 ± 0.4	**87.54 ± 0.6**	74.38 ± 0.9
Multi 4	76.82 ± 0.3	87.43 ± 0.4	76.05 ± 0.6

Input	Rec_C	Rec_H	Rec_E
Multi 2	78.57 ± 0.4	**81.08 ± 0.9**	77.02 ± 0.9
Multi 3	78.61 ± 0.4	79.42 ± 0.9	**78.07 ± 0.9**
Multi 4	**79.81 ± 0.3**	78.97 ± 0.7	75.76 ± 0.6

Input	F-measure$_C$	F-measure$_H$	F-measure$_E$	Acc
Multi 2	**77.66 ± 0.3**	**84.80 ± 0.5**	79.06 ± 0.5	**82.99 ± 0.3**
Multi 3	77.39 ± 0.3	84.44 ± 0.6	78.87 ± 0.5	82.61 ± 0.3
Multi 4	77.60 ± 0.2	84.11 ± 0.4	**78.59 ± 0.4**	82.68 ± 0.2

TABLE 3.4: Class-specific measurement results and accuracy averaged over the proteins of the test set after 100 epochs with different MTL variants. The PSSM+ProtVec input described in Section 2.2.1 is used. The abbreviations are: Multi 2 (DSSP3 & RSA), Multi 3 (DSSP3, RSA & B-Factors) and Multi4 (DSSP3, DSSP8, RSA & B-Factors). The best values per column are highlighted.

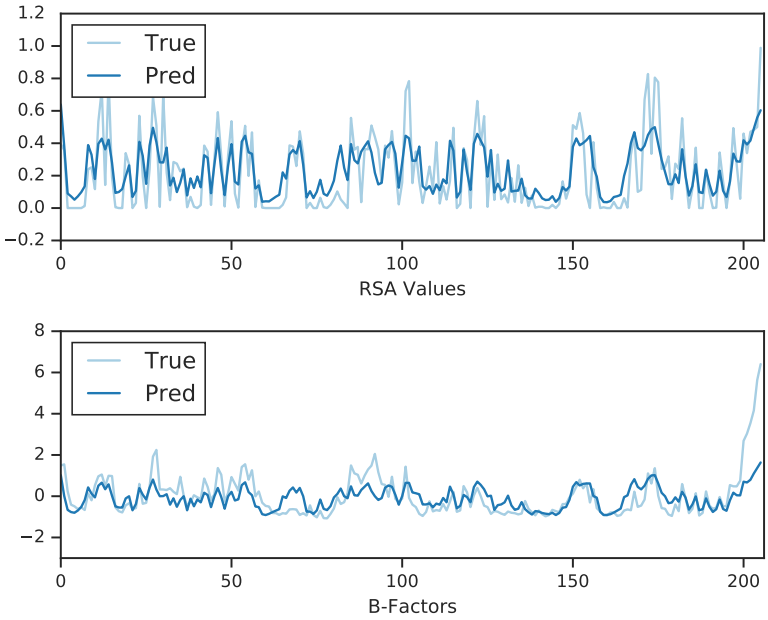


FIGURE 3.6: Predictions of RSA values (top) and B-factors (bottom) for the randomly chosen protein ITU8 with MTL of the DSSP3 structures, the RSA values and the B-factors. The DenseHybrid network described in Section 2.2.2 was used. The residues of the protein are displayed along the x-axis and the predicted values are shown on the y-axis at each position.

3.2.4 Effect of Mapping Predictions from DSSP8 to DSSP3 Structures

In a further experiment, we investigate the effect of first learning DSSP8 states and then mapping the resulting predictions to DSSP3 states according to Table 1.1. Again, the DenseHybrid architecture and the PSSM+ProtVec input are used, as well as multi-task learning of DSSP3 structures and RSA values. The class-specific measurement results and the accuracy can be seen in Table 3.5. We can observe no improvement over the direct learning of DSSP3 structures but merely a small reduction of 0.81%. Note here that no class balancing was employed because the main task was the DSSP8 classification.

$Prec_C$	$Prec_H$	$Prec_E$	Rec_C	Rec_H	Rec_E
77.85 ± 0.4	86.45 ± 0.6	73.80 ± 0.9	76.13 ± 0.4	78.67 ± 1.0	77.79 ± 0.9

F-measure$_C$	F-measure$_H$	F-measure$_E$	Acc
76.37 ± 0.3	84.96 ± 0.5	78.42 ± 0.5	82.18 ± 0.3

TABLE 3.5: Class-specific measurement results and accuracy after training the DenseHybrid network described in Section 2.2.2 on DSSP8 structures and mapping the result to DSSP3 structures. The PSSM+ProtVec representation described in Section 2.2.1 and MTL of the DSSP3 structures and RSA values is used. The measurement results are averaged over the proteins in the test set.

3.2.5 DSSP8 Classification

Despite that we focus in this thesis on DSSP3 classification, it is interesting to see how the previously best performing model − the DenseHybrid model with multi-task learning of DSSP3 structures and RSA values, together with the PSSM+ProtVec input − performs for DSSP8 classification. This task is more challenging than the DSSP3 classification, not only because of the more specific classes, but also because of the very unbalanced class distribution. Like in the preceding experiment, no class balancing was used.

The learning progress is shown in Figure 3.7. We can observe that the Q8 accuracy converges at 70.48%. The confusion matrix of the test set after 100 epochs is shown in Figure 3.8 and the class-specific performance measures are summarized in Table 3.6. We can see that classes I and B are never predicted and that very few predictions of classes S and G exist. Note here furthermore that the precision standard error of the two latter classes is particularly high.

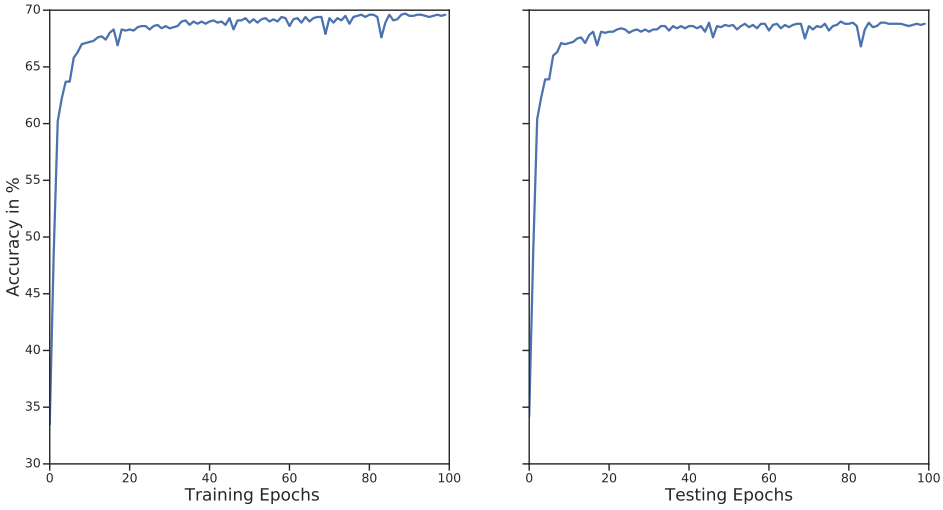


FIGURE 3.7: Learning curve of training (left) and testing (right) for the DenseHybrid network described in 2.2.2 for DSSP8 classification. The PSSM+ProtVec input described in Section 2.2.1 and MTL of DSSP8 structures and RSA values are used.

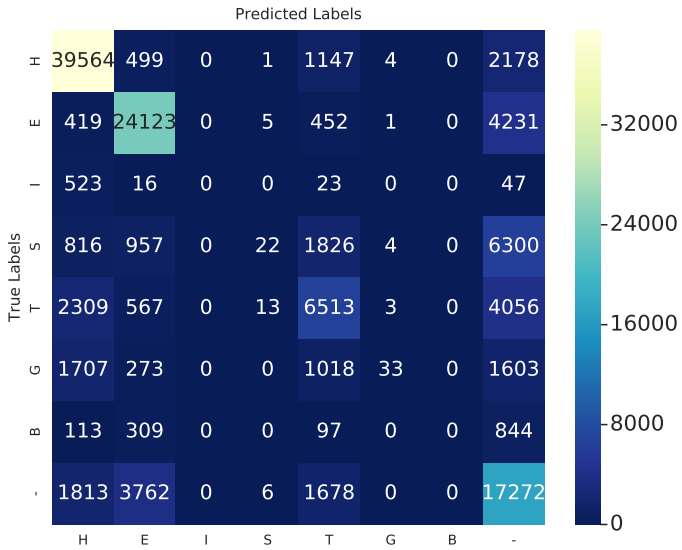


FIGURE 3.8: Confusion matrix on the test set after 100 epochs for the DenseHybrid network described in Section 2.2.2 for DSSP8 classification. The PSSM+ProtVec representation described in Section 2.2.1 and MTL of the DSSP8 structures and the RSA-values are used. The values state the absolute counts of observed samples in each class.

$Prec_H$	$Prec_E$	$Prec_S$	$Prec_T$	$Prec_G$	$Prec_-$
80.50 ± 0.8	74.92 ± 1.0	45.93 ± 7.7	51.92 ± 0.8	70.14 ± 7.5	50.09 ± 0.5

Rec_H	Rec_E	Rec_S	Rec_T	Rec_G	Rec_-
88.94 ± 0.7	81.64 ± 0.7	0.21 ± 0.6	49.72 ± 0.8	0.86 ± 0.2	70.48 ± 0.5

F_H	F_E	F_S	F_T	F_G	F_-
85.68 ± 0.4	80.14 ± 0.5	10.98 ± 1.8	50.44 ± 0.6	25.57 ± 3.2	57.49 ± 0.4

Accuracy
70.48 ± 0.4

Table 3.6: Class-specific measurement results and accuracy averaged over the proteins in the test set after 100 epochs for the DenseHybrid network described in Section 2.2.2 for DSSP8 classification. The PSSM+ProtVec representation described in Section 2.2.1 and the MTL of the DSSP8 structures and the RSA-values are used. The results for classes I and B are not given as they were never predicted and thus result in NaN/0.0.

3.2.6 Effect of ProtVec for the DenseHybrid Architecture

Finally, we extend the findings for the CNN of the experiment in Section 3.2.1 to the best approach determined so far which includes multi-task learning, namely to the DenseHybrid architecture in combination with the PSSM+ProtVec input and multi-task learning of DSSP3 structures and RSA values. We feed it with all five input representations. Figure 3.9 shows the development of the performance over 100 epochs.

Here, we can observe a similar behavior as for the CNN in Section 3.2.1. An increase in accuracy of $\geq$ 11.46% can be observed when including evolutionary information. In contrast to the experiment with the CNN, the One-hot encoding outperforms the plain ProtVec representation already after about seven epochs. After 100 epochs, the use of the ProtVec representation results in 5.16% less accuracy. The performance for the PSSM+ProtVec input is 0.17% higher than for the PSSM matrix and reaches 82.99% accuracy. Compared to the setup of Section 3.2.1, we can observe a marginal increase of 0.37% and 0.61% for the PSSM and the PSSM+ProtVec input, respectively. The ProtVec representation of the MSA achieves only 2.54% accuracy less than the PSSM+ProtVec input here, whereas this difference is 4.47% for the CNN network, compare Table 3.2. Finally, the best Q3 accuracy reached by the DenseHybrid network with this multi-task learning variant is 82.99%, which is 0.18% lower than without multi-task learning.

Altogether, the best results in our experiments were achieved by the DenseHybrid architecture with plain structure prediction and also when employing multi-task learning of structures and RSA values.

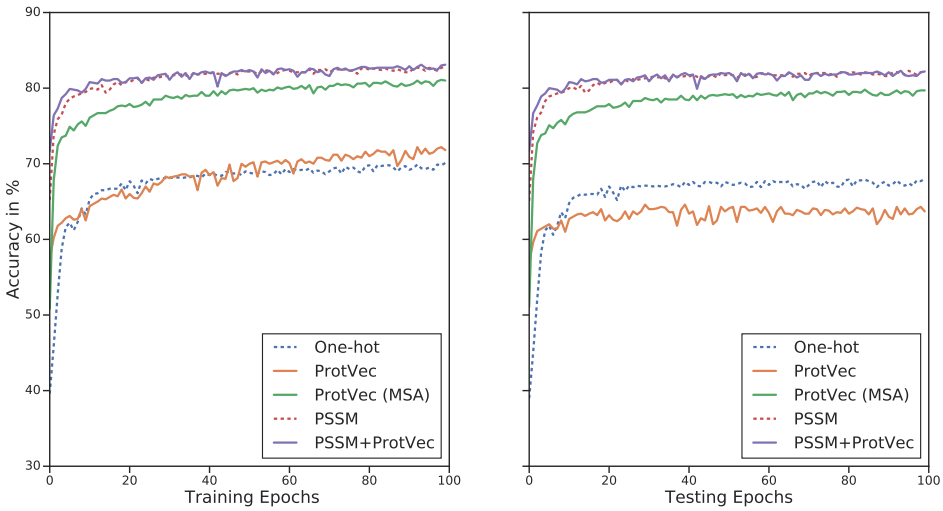


FIGURE 3.9: Learning curves of training (left) and testing (right) the DenseHybrid network of Section 2.2.2 on different inputs for 100 epochs. DSSP3 structures are learned and MTL of the DSSP3 structures and RSA-values is used. The input representations are given in Section 2.2.1.

One-hot	ProtVec	ProtVec (MSA)	PSSM	PSSM+ProtVec
68.99 ± 0.3	64.83 ± 0.4	80.45 ± 0.3	82.82 ± 0.3	$\mathbf{82.99 \pm 0.3}$

TABLE 3.7: Accuracies averaged over the proteins in the test set after 100 epochs for the DenseHybrid network. The PSSM+ProtVec input and MTL of DSSP3 structures and RSA values are used.

3.3 Runtime and Number of Parameters

All computations are done on a GeForce GTX 1080 GPU. Table 3.8 shows a summary of the runtime as well as the number of learnable parameters of each network architecture. The runtime is based on 100 epochs and takes the training and testing into account. All networks are evaluated on exactly the same training and test set.

It is important to note that the choice of the input representation contributes to the number of parameters. The first layer of a neural network needs to be adjusted to the size of the network input and thus has different numbers of parameters for different input representations. Table 3.8 is based on taking PSSM+ProtVec inputs. We can see that (by choice of network design, compare Section 2.2.2) the number of parameters are about the same for all six network architectures. The

runtimes however, differ substantially for the different architectures. For example the DenseHybrid network has a runtime twice as long as that of the CNN and the biLSTM has a runtime three times as long as that of the DenseHybrid network. Altogether, the DenseHybrid architecture is the best choice with respect to accuracy as it achieves the highest accuracies in our experiments. In contrast, the CNN and DenseCNN architectures are the best choices with respect to speed (and still show quite good performance).

Network	Runtime	Parameters
CNN	355.59 s	28312
DenseCNN	354.092 s	28957
LSTM	1008.693 s	30911
biLSTM	1902.309 s	26287
Hybrid	850.538 s	29391
DenseHybrid	644.909 s	28809

Table 3.8: Number of parameters and runtime for each network, based on operating on the PSSM+ProtVec input. The runtime is given in seconds.

Chapter 4

Discussion

In this section we summarize and discuss our findings.

Evolutionary Information First of all, our results confirm the importance of evolutionary information in secondary structure prediction. Using the raw PSSM matrix instead of the One-hot encoding results in an increase of 15.48% and 13.83% accuracy for the CNN and DenseHybrid network, respectively.

ProtVec embeddings Our experiments with the CNN in Section 3.2.1 show that no improvement can be achieved by the use of ProtVec embeddings as an input. Surprisingly, the ProtVec input and the One-hot input share the same performance for the first 50 epochs, then the performance on ProtVec does not increase further and, after 100 epochs, gives an accuracy of the CNN which is 2.52% lower than that for the One-hot input. Furthermore, our experiment in Section 3.2.1 shows that the performance in the first 20 epochs is slightly better for the PSSM+ProtVec input than for the raw PSSM matrix. However, with more epochs, both input types converge to an equal performance of about 82.4%. A reason for this behavior might be that the information on biochemical properties, transferred from the SwissProt database to the given data set through the ProtVec embeddings, is already learned by the CNN after 20 epochs. Our experiment in Section 3.2.6 for the DenseHybrid network shows a similar long-term behavior for the different inputs. The One-hot input outperforms the ProtVec representation with an even higher difference of accuracy than for the CNN, namely 4.16%, and the improvement of 0.17% accuracy on the PSSM+ProtVec input over the raw PSSM is insignificantly small. The ProtVec encoding of the MSAs without the PSSM matrix achieves 4.54% and 2.37% less accuracy than that with the PSSM matrix for the CNN and DenseHybrid, respectively. Here, the additional runtime of computing the PSSM matrix has to be kept in mind. To sum up, simple ProtVec embeddings do not yield any increase in performance over inputs which do not use them. However, the combination of ProtVec matrices of the MSAs and the PSSM matrices can result in a slightly increased performance.

Network architecture Comparing the performance of different network architectures of our experiments in Section 3.2.2 shows that the choice of network type does not have a significant impact on the performance, as long as it can access positions to both sides of a residue. All networks converge to accuracies in the range

of 82.22% − 83.17%, except for the LSTM which reaches only 78.28%. This reduced performance can be explained with the fact that the LSTM is the only network which strictly works in a uni-directional manner along the input sequence. All other networks have access to residues at later positions in the sequence to some extent. The contexts to both sides of one residue hold important information for the secondary structure at that position. This was to be expected anyway.

Multi-Task learning Our experiments in Section 3.2.3 show that the DenseHybrid network does not profit significantly from multi-task learning. This is independent of the choice of targets to be learned simultaneously. However, we see that the network can learn additional information without loosing significant performance. Figure 3.6 supports this result. Compared to only learning the protein structure, the learning of all four targets simultaneously results in a marginal decrease of 0.49% accuracy. Learning only the RSA values besides the structures results in a decrease of 0.18% accuracy. Interestingly, the latter approach is minimally better than additionally learning the B-factors. This might be a result of the fact that more than 70% of the residues have a B-factor value between -1 and 1 (see Figure 2.4b) or might stem from the specific way the B-factors were generated.

Mapping DSSP8 states to DSSP3 states Our results from Section 3.2.4 show that learning DSSP8 state targets and then mapping the prediction to DSSP3 targets does not increase but slightly decrease the performance from 82.88% accuracy to 81.24% accuracy. Thus, this procedure is not recommended. One has to keep in mind that this approach did not use weight balancing, in contrast to the direct prediction of the DSSP3 structures.

DSSP8 classification Our experiment of Section 3.2.5 confirms that learning the more detailed DSSP8 secondary structures remains a challenge. Most likely, the issues emerge from the class imbalance of the eight states, which does not allow the DenseHybrid network to predict the underrepresented states at all. In our results it only predicts the states H, E, T, −, and in very few cases the states S and G while the states I and B were never predicted.

Best performing model The best performance for DSSP3 classification on the given data set in this thesis is an accuracy of 83.17% ± 0.3. This is achieved by the DenseHybrid architecture and the PSSM+ProtVec representation without using multi-task learning. In comparison to the number of parameters of NetSurfP2 (around 35.000.000), which presently achieves 85% accuracy, the number of parameters of our DenseHybrid architecture is very small (28.809). We found that, if desired, all four structural targets can be learned without a significant loss. The gain of information outweighs the small loss in performance and thus the DenseHybrid model together with the multi-task variant which learns all four targets simultaneously shall be considered the best model of this thesis. Another important aspect is given by the runtimes of the individual networks. Table 3.8 shows that the different networks, despite possessing roughly the same amount of parameters, have significantly different runtimes.

LSTMs are known to have a higher computational complexity than CNNs due to their sequential nature, which is confirmed by our results. The biLSTM, DenseCNN, Hybrid and DenseHybrid architectures roughly perform equally well (compare Table 3.3). If runtime is the limiting factor, the CNN and the DenseCNN architectures without multi-task learning are the best choice, achieving just 0.79% and 0.95% less accuracy, respectively, of the DenseHybrid architecture with half of the runtime on the PSSM+ProtVec input.

Chapter 5

Conclusion and Outlook

In this thesis, we have examined the performance of six different network architectures on five input representations in order to evaluate the effect of encoding protein sequences through ProtVec word embeddings in protein secondary structure prediction. The expectation for ProtVec embeddings to unfold the potential of Word2Vec not only in NLP but also in this field could not be met. It is doubtful if structure prediction can benefit from ProtVec in another way. A method which does not rely on the expensive computation of MSAs but predicts the structure well is still to be found. Until there is an alternative, the trade-off between performance and computational complexity remains a central problem in protein secondary structure prediction. It became clear that the choice of input features is significantly more important than the choice of the network architecture as long as the network can access neighboring residues to both sides of the positions. Our experiments further suggested that learning additional structural features is possible by using multi-task learning, without loosing significant performance. Altogether, a Q3 accuracy of 83.17% could be reached by our simple and efficient DenseHybrid architecture when the PSSM matrix concatenated with the ProtVec encoding of the MSA serves as the input.

Since several hyperparameters can be chosen for Word2Vec, it could be tested in the future if longer n-grams or a bigger context size might change the outcome. However, this seems unlikely. It could also be investigated what effect the second commonly used method for generating word embeddings, called GloVe [49], would have in the context of proteins. This method directly captures the global statistics of the word-word co-occurences of a corpus. Furthermore, it could be studied if padding not only to one side of the amino acid sequences, but to both sides, or the use of an alternative class balancing might yield an increase in performance. It might also be a good idea to further study the network architectures suggested in NLP. Here, attention models learn to place more weight on important parts of the input data. Google AI has introduced the Transformer model [50] which only relies on attention mechanisms. Based on this, the BERT network [51] has been proposed, operating bidirectionally. Both networks could be interesting to look at in the future.

Bibliography

[1] E. Asgari and M. Mofrad, "ProtVec: A continuous distributed representation of biological sequences for deep proteomics and genomics", *PLoS ONE*, vol. 10, no. 11, 2015.

[2] H. Berman, J. Westbrook, et al., "The protein data bank", *Nucleic acids research*, vol. 28, no. 1, pp. 235–242, 2000, accessed 23 Oct 2018. [Online]. Available: `www.rcsb.org`.

[3] I. Kuntz, "Structure-based strategies for drug design and discovery", *Science*, vol. 257, no. 5073, pp. 1078–1082, 1992.

[4] A. Nuccitelli, R. Cozzi, et al., "Structure-based approach to rationally design a chimeric protein for an effective vaccine against group B streptococcus infections", *Proceedings of the National Academy of Sciences*, vol. 108, no. 25, pp. 10 278–10 283, 2011.

[5] M. Krishna and G. Khushboo, *Protein Engineering Techniques*. Springer, 2017, pp. 61–83.

[6] *UniProtKB/Swiss-Prot UniProt release 2018-11*, Accessed: 2018-12-12. [Online]. Available: `https://www.uniprot.org/statistics/Swiss-Prot`.

[7] *PDB statistics: Overall growth of released structures per year*, Accessed: 2018-12-12. [Online]. Available: `https://www.rcsb.org/stats/growth/overall`.

[8] B. Rost and C. Sander, "Prediction of protein secondary structure at better than 70% accuracy", *Journal of Molecular Biology*, vol. 232, no. 2, pp. 584–599, 1993.

[9] Z. Li and Y. Yu, "Protein secondary structure prediction using cascaded convolutional and recurrent neural networks", in *Proceedings of the Twenty-Fifth International Joint Conference on Artificial Intelligence*, S. Kambhampati, Ed., AAAI Press, New York, 2016, pp. 2560–2567.

[10] B. Zhang, J. Li, and Q. Lü, "Prediction of 8-state protein secondary structures by a novel deep learning architecture", *BMC Bioinformatics*, vol. 19, 2018.

[11] M. Torrisi, M. Kaleel, and G. Pollastri, "Porter 5: Fast, state-of-the-art ab initio prediction of protein secondary structure in 3 and 8 classes", *bioRxiv*, 2018. [Online]. Available: `https://www.biorxiv.org/content/early/2018/10/05/289033`.

[12] M. Klausen, M. Jespersen, et al., "NetSurfP-2.0: Improved prediction of protein structural features by integrated deep learning", *Proteins: Structure, Function, and Bioinformatics*, 2019, in print.

[13] C. Anfinsen, "Principles that govern the folding of protein chains", *Science*, vol. 181, no. 4096, pp. 223–230, 1973.

[14] L. Pauling, R. Corey, and H. Branson, "The structure of proteins: Two hydrogen-bonded helical configurations of the polypeptide chain", *Proceedings of the National Academy of Sciences*, vol. 37, no. 4, pp. 205–211, 1951.

[15] W. Kabsch and C. Sander, "Dictionary of protein secondary structure: Pattern recognition of hydrogen-bonded and geometrical features", *Biopolymers: Original Research on Biomolecules*, vol. 22, no. 12, pp. 2577–2637, 1983.

[16] M. Tien, A. Meyer, et al., "Maximum allowed solvent accessibilites of residues in proteins", *PloS one*, vol. 8, no. 11, e80635, 2013.

[17] A. Momen-Roknabadi, M. Sadeghi, et al., "Impact of residue accessible surface area on the prediction of protein secondary structures", *BMC bioinformatics*, vol. 9, no. 1, p. 357, 2008.

[18] H. Zhang, T. Zhang, et al., "On the relation between residue flexibility and local solvent accessibility in proteins", *Proteins: Structure, Function, and Bioinformatics*, vol. 76, no. 3, pp. 617–636, 2009.

[19] M. Steinegger and J. Söding, "MMseqs2 enables sensitive protein sequence searching for the analysis of massive data sets", *Nature Biotechnology*, vol. 35, no. 11, p. 1026, 2017.

[20] S. Altschul, T. Madden, et al., "Gapped BLAST and PSI-BLAST: A new generation of protein database search programs", *Nucleic acids research*, vol. 25, no. 17, pp. 3389–3402, 1997.

[21] M. Remmert et al., "HHblits: Lightning-fast iterative protein sequence searching by HMM-HMM alignment", *Nature methods*, vol. 9, no. 2, p. 173, 2012.

[22] C. Bishop, *Pattern Recognition and Machine Learning (Information Science and Statistics)*. Berlin, Heidelberg: Springer-Verlag, 2006.

[23] S. Ren, K. He, et al., "Faster r-cnn: Towards real-time object detection with region proposal networks", in *Advances in Neural Information Processing Systems*, 2015, pp. 91–99.

[24] D. Palaz, R. Collobert, et al., "Analysis of cnn-based speech recognition system using raw speech as input", in *Proceedings of INTERSPEECH, 16th Annual Conference of the International Speech Communication Association*, Dresden, 2015.

[25] D. Tran, L. Bourdev, et al., "Learning spatiotemporal features with 3d convolutional networks", in *Proceedings of the IEEE international conference on computer vision (ICCVW)*, Santiago, Chile, 2015, pp. 4489–4497.

[26] M. Defferrard, X. Bresson, and P. Vandergheynst, "Convolutional neural networks on graphs with fast localized spectral filtering", in *Advances in Neural Information Processing Systems*, 2016, pp. 3844–3852.

[27] G. Huang, Z. Liu, et al., "Densely connected convolutional networks.", in *CVPR*, vol. 1, 2017, p. 3.

[28] S. Hochreiter and J. Schmidhuber, "Long short-term memory", *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[29] M. Schuster and K. Paliwal, "Bidirectional recurrent neural networks", *IEEE Transactions on Signal Processing*, vol. 45, no. 11, pp. 2673–2681, 1997.

[30] I. Sutskever, O. Vinyals, and Q. Le, "Sequence to sequence learning with neural networks", in *Advances in Neural Information Processing Systems (NIPS 2014)*, 2014, pp. 3104–3112.

[31] A. Graves, S. Fernández, and J. Schmidhuber, "Bidirectional LSTM networks for improved phoneme classification and recognition", in *International Conference on Artificial Neural Networks*, Springer, 2005, pp. 799–804.

[32] J. Chiu and E. Nichols, "Named entity recognition with bidirectional LSTM-CNNs", *Transactions of the Association for Computational Linguistics*, vol. 4, pp. 357–370, 2016.

[33] M. Stollenga, W. Byeon, et al., "Parallel multi-dimensional LSTM, with application to fast biomedical volumetric image segmentation", in *Advances in Neural Information Processing Systems (NIPS 2015)*, 2015, pp. 2998–3006.

[34] R. Caruana, "Multitask learning", *Machine learning*, vol. 28, no. 1, pp. 41–75, 1997.

[35] R. Ranjan, V. Patel, and R. Chellappa, "Hyperface: A deep multi-task learning framework for face detection, landmark localization, pose estimation, and gender recognition", *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 41, no. 1, pp. 121–135, 2019.

[36] B. Ramsundar, S. Kearnes, et al., "Massively multitask networks for drug discovery", *arXiv preprint arXiv:1502.02072*, 2015. [Online]. Available: `https://arxiv.org/abs/1502.02072`.

[37] T. Mikolov, I. Sutskever, et al., "Distributed representations of words and phrases and their compositionality", in *Advances in Neural Information Processing Systems*, 2013, pp. 3111–3119.

[38] T. Mikolov, K. Chen, et al., "Efficient estimation of word representations in vector space", *Proceedings of Workshop at ICLR*, Jan. 2013.

[39] UniProt Consortium, "Uniprot: A hub for protein information", *Nucleic acids research*, vol. 43, no. D1, pp. D204–D212, 2014.

[40] H. Bohr, J. Bohr, et al., "A novel approach to prediction of the 3-dimensional structures of protein backbones by neural networks", *FEBS letters*, vol. 261, no. 1, pp. 43–46, 1990.

[41] L. Holley and M. Karplus, "Protein secondary structure prediction with a neural network", *Proceedings of the National Academy of Sciences*, vol. 86, no. 1, pp. 152–156, 1989.

[42] N. Qian and T. Sejnowski, "Predicting the secondary structure of globular proteins using neural network models", *Journal of Molecular Biology*, vol. 202, no. 4, pp. 865–884, 1988.

[43] R. Heffernan, Y. Yang, et al., "Capturing non-local interactions by long short-term memory bidirectional recurrent neural networks for improving prediction of protein secondary structure, backbone angles, contact numbers and solvent accessibility", *Bioinformatics*, vol. 33, no. 18, pp. 2842–2849, 2017.

[44] Z. Wang, F. Zhao, et al., "Protein 8-class secondary structure prediction using conditional neural fields", *Proteomics*, vol. 11, pp. 3786–92, 2011.

[45] W. Touw and G. Vriend, "BDB: Databank of PDB files with consistent B-factors", *Protein Engineering, Design & Selection*, vol. 27, no. 11, pp. 457–462, 2014.

[46] D. Kingma and J. Ba, "Adam: A method for stochastic optimization", in *Proceedings of Workshop at ICLR*, San Diego, 2015.

[47] S. Reddi, S. Kale, and S. Kumar, "On the convergence of Adam and beyond", in *Proceedings of Workshop at ICLR*, Vancouver, 2018.

[48] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.

[49] J. Pennington, R. Socher, and C. Manning, "GloVe: Global vectors for word representation.", Doha, Qatar, 2014.

[50] A. Vaswani, N. Shazeer, et al., "Attention is all you need", in *Advances in Neural Information Processing Systems 30*, I. Guyon, U. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds., Curran Associates, Inc., 2017, pp. 5998–6008.

[51] J. Devlin, M.-W. Chang, et al., "BERT: Pre-training of deep bidirectional transformers for language understanding", *arXiv preprint arXiv:1810.04805*, 2018.

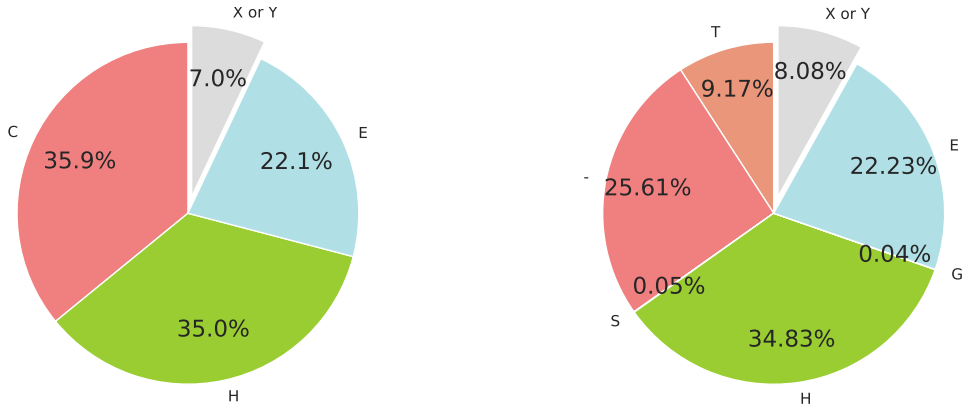
Appendix A

Supplementary Material

A.1 Data Analysis of the Predicted DSSP3 Structures

In the following, the *predicted* structure states from the DenseHybrid network with the multi-task learning of DSSP structures and RSA values are analyzed. The PSSM+ProtVec input was used. This is done for DSSP3 classification (see Section 3.2.3) and DSSP8 classification (see Section 3.2.5). The results are then compared to the a priori data analysis of Section 2.1.

The class distribution for the predicted states are shown in Figure A.1. We can observe that in DSSP3 classification the occurrence of the three states is around the same as in the original structure targets. Just states C and E occur slightly more often here. In DSSP8 classification, only states T, S, H, G, E and − are predicted.



(A) DSSP3 (B) DSSP8

FIGURE A.1: Distribution of the DSSP states in the *predicted* structures. The left side shows the DSSP3 classification and the right side the DSSP8 classification. The state-abbreviations are as given in Table 1.1. State X and Y are auxiliary states of positions where no measurements could be made and positions where the DSSP algorithm determined different states in the different sequence chains, respectively.

These states all occur more often than in the original targets. The 'X and Y' state occurs less often in both, the predicted DSSP3 and the predicted DSSP8 states. The distribution of lengths of sub-sequences of the same structure state is shown in Figure A.2.

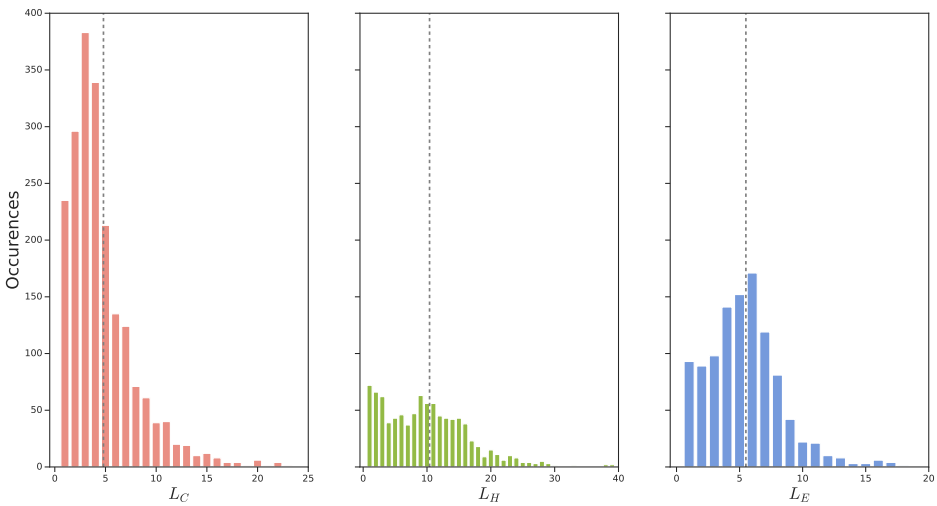


FIGURE A.2: Distribution of lengths of sub-sequences with only one structure state in the *predicted* structures. L_c refers to the length of a sub-sequence consisting solely of state c, given in numbers of positions. The x-axis shows the lengths and the y-axis shows the number of occurrences. The average lengths are indicated by the dashed lines. Outliers are cut off for visual reasons. These are L_H of 61, 49 and 45 as well as L_C of 30, 27 and 26.

In DSSP3 classification, structure C occurs five times, state H occurs ten times and state E occurs six times in a row on average, which again is higher for states C and E, here in comparison to the original targets. Moreover, the average distance between two β-sheets is the same for the original and the predicted structures, namely five.

A.2 Loss Curves

In the following, the corresponding loss curves of the learning curves of our experiments described in Chapter 3 are given.

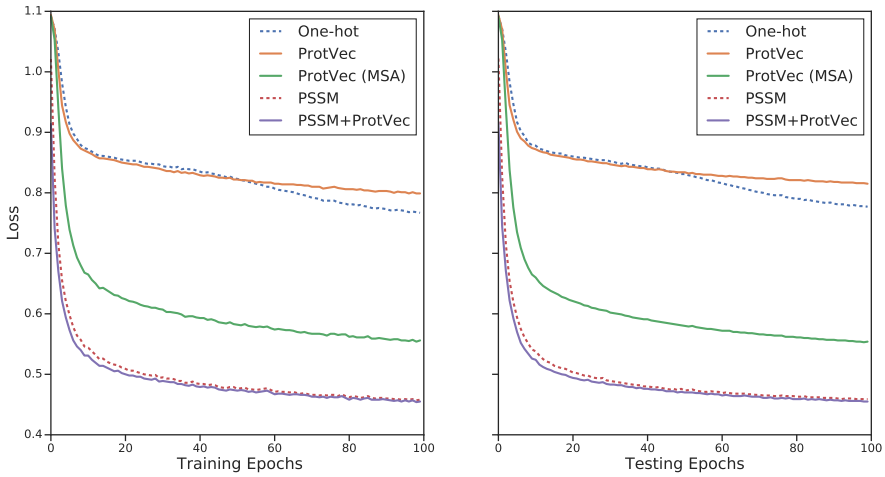


FIGURE A.3: Loss curves of the CNN for different inputs, corresponding to Figure 3.1.

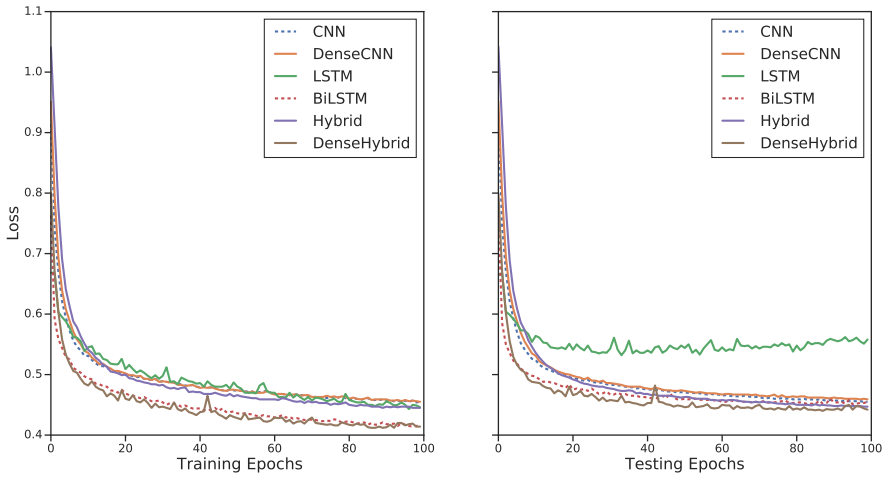


FIGURE A.4: Loss curves of different network architectures for PSSM+ProtVec input, corresponding to Figure 3.4.

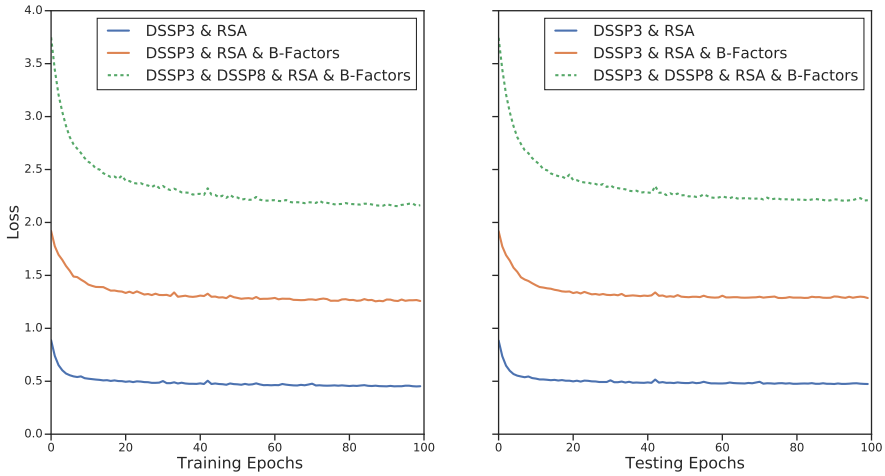


FIGURE A.5: Loss curves of the DenseHybrid architecture for different MTL variants, corresponding to Figure 3.5. The PSSM+ProtVec input was used. Note that the loss shown is the final multi-target loss which is combined through summation of the losses of the individual tasks.

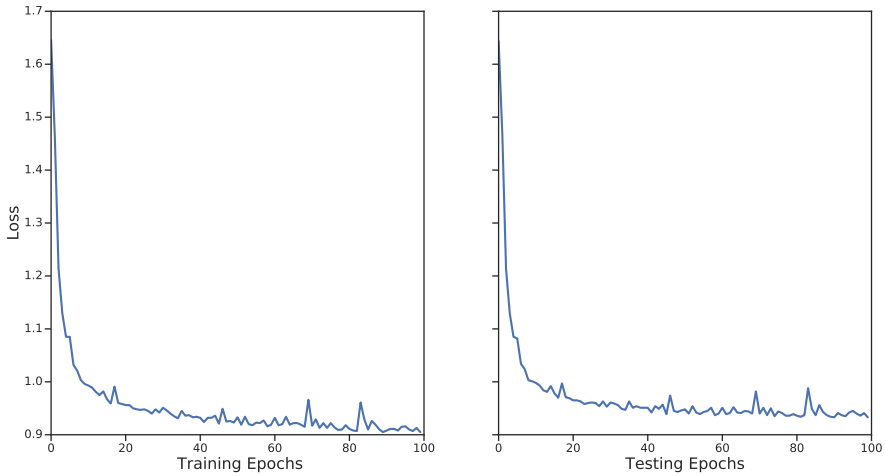


FIGURE A.6: Loss curve of the DenseHybrid architecture for DSSP8 classification, corresponding to Figure 3.7. The PSSM+ProtVec input was used.

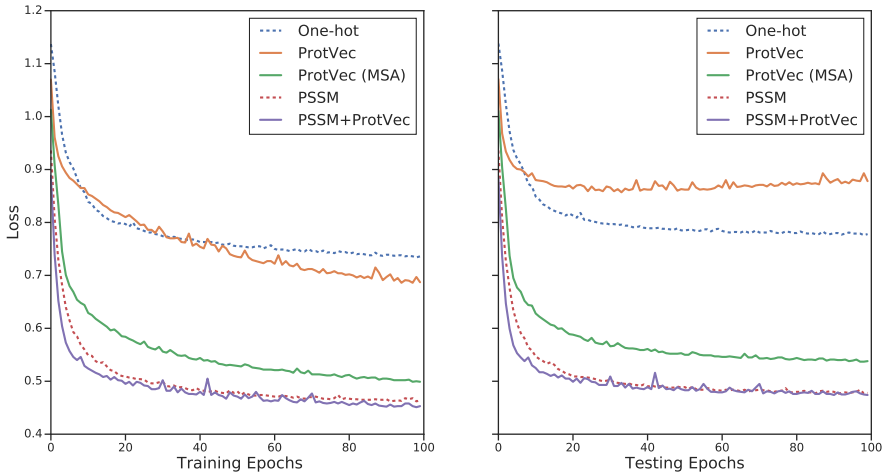


FIGURE A.7: Loss curves of the DenseHybrid architecture for different inputs, corresponding to Figure 3.9.